# Metodologie di Programmazione (A-L)

Il semestre - a.a. 2023 – 2024

Parte 14 – Design Pattern**

a cura di Mattia Samory*




SAPIENZA
UNIVERSITÀ DI ROMA

*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

# Letture

- SO: Examples of GoF Design Patterns in Java's core libraries https://stackoverflow.com/questions/1673841/examples-of-gof-design-patterns-in-javas-core-libraries


NAH,
I'D
WIN.

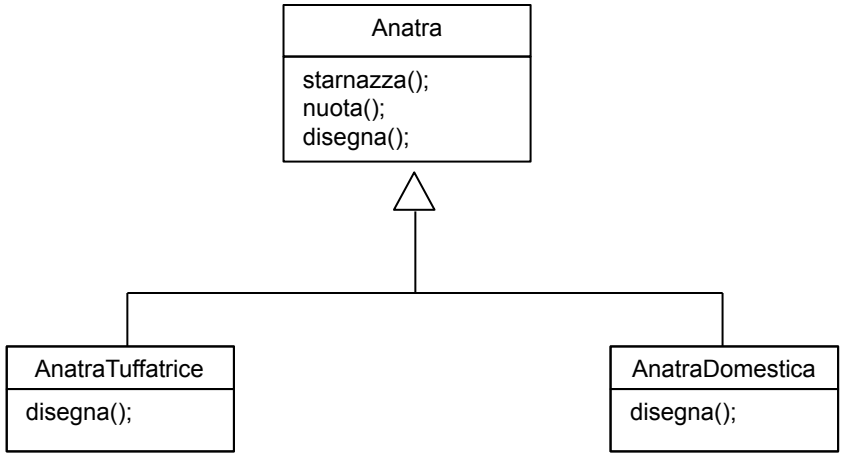

NAH,
I'D
WIN.

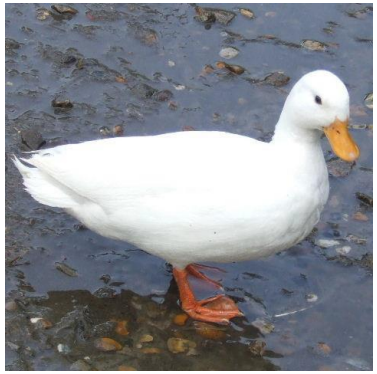
# Iniziamo con un caso concreto: un Simulatore di Anatre

- Hai appena sviluppato un fantastico simulatore di anatre: IDuck
- Hai strutturato le classi in questo modo:





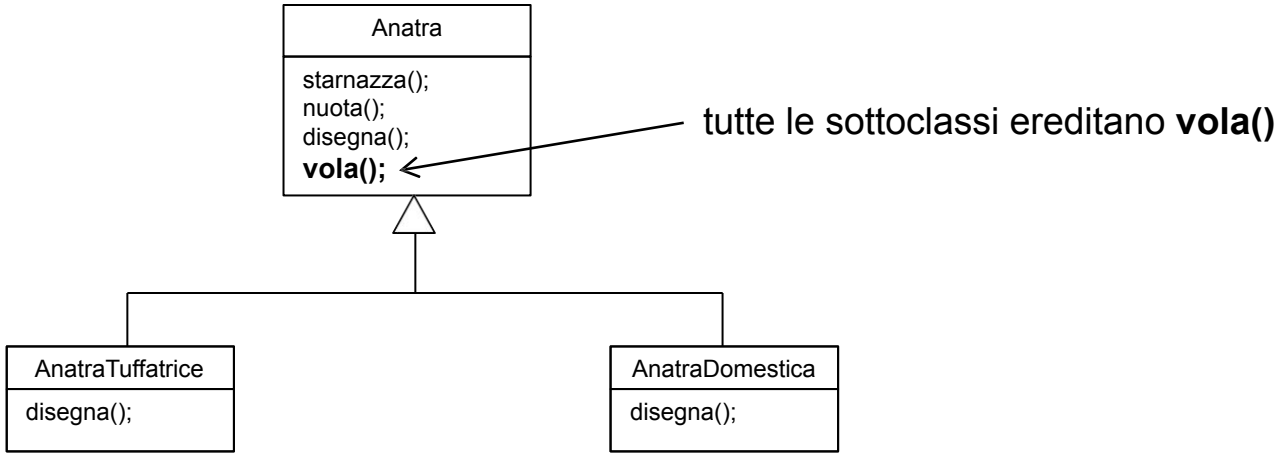


# Ma ora vogliamo aggiungere il volo!

- Il cliente decide che il simulatore deve permettere alle anatre di volare
- Stiamo programmando a oggetti: quanto può essere difficile?

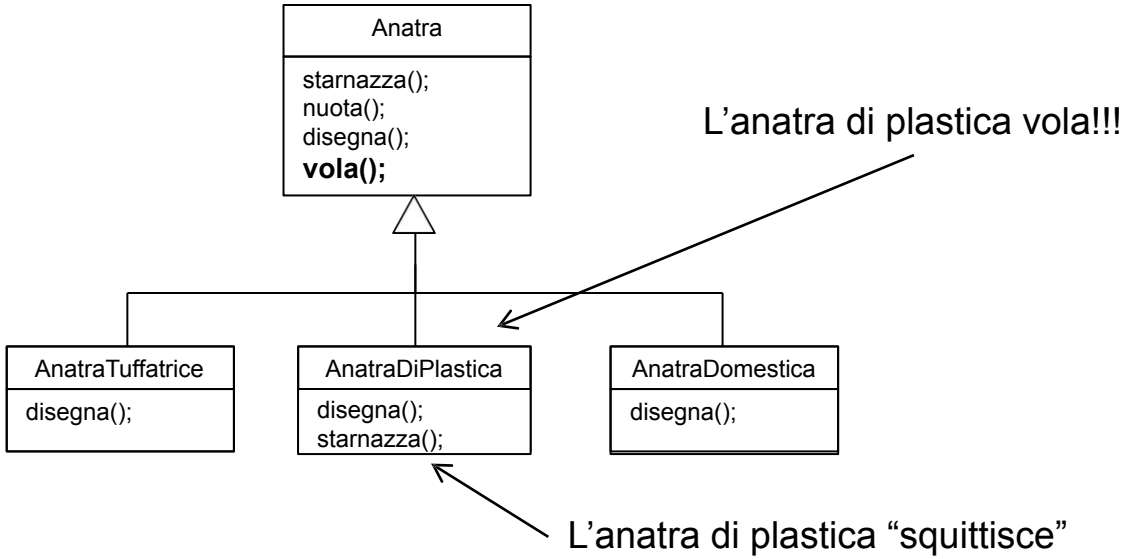




# Ma qualcosa può andare male!

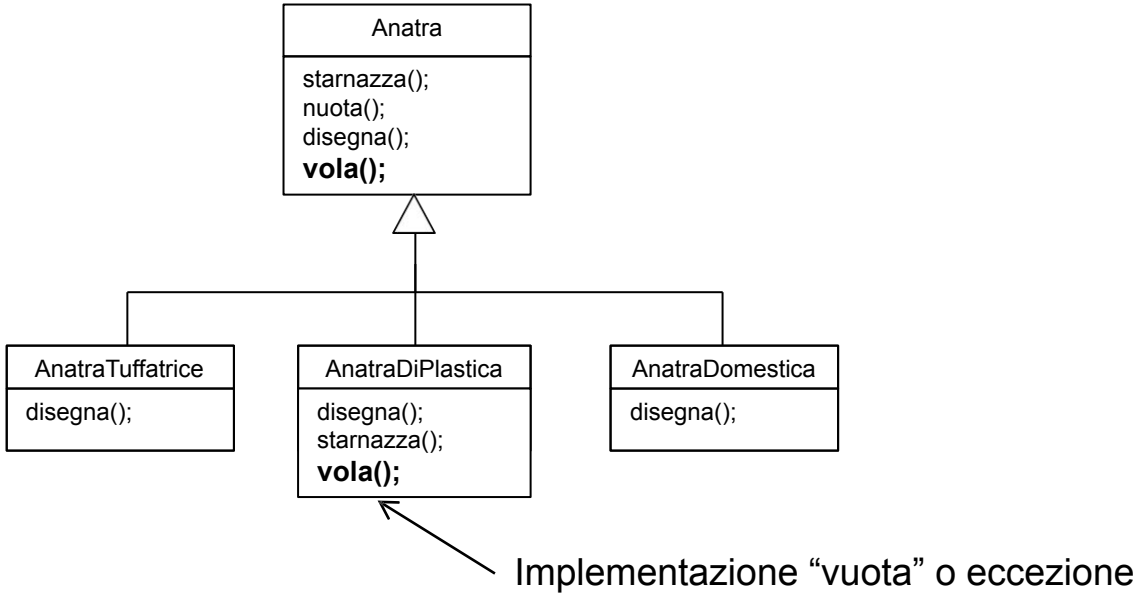


- Anche le anatre di plastica volano!
- Nell'aggiungere il metodo vola non abbiamo pensato al fatto che non tutte le anatre volano (!)
- L'ereditarietà è utile per il riuso, ma meno per la manutenzione!



# Come risolvere il problema?

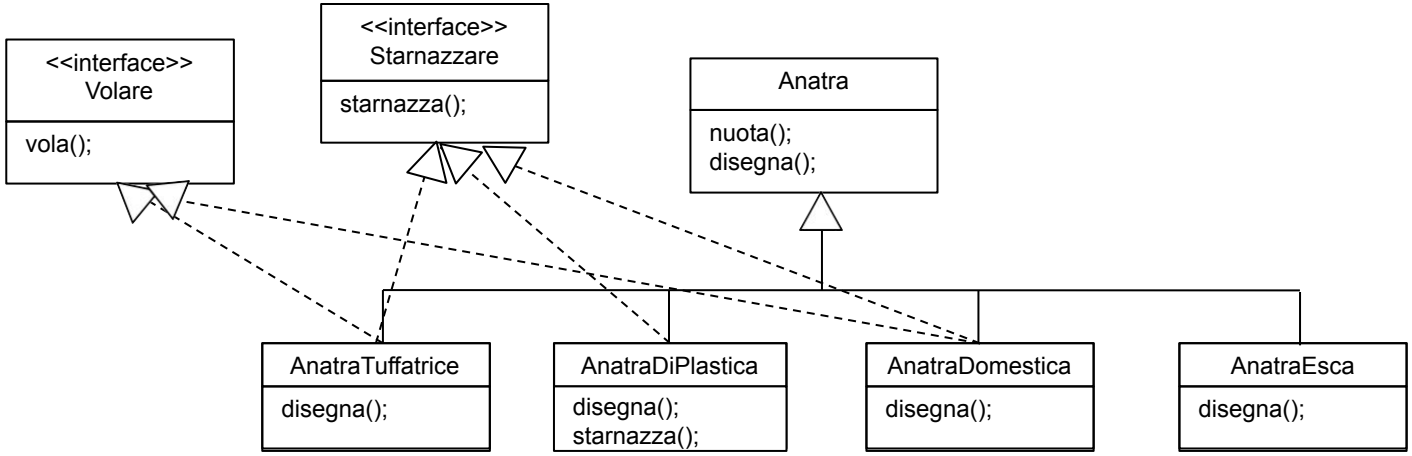
- **Idea 1:** Sovrascriviamo il metodo vola nell'AnatraDiPlastica!




- Ma se aggiungiamo un altro tipo di anatra, l'AnatraEsca?
- Le anatre esca sono esche: non volano né starnazzano…

# E se usassimo le interfacce?

- **Idea 2:** implementiamo un'interfaccia per ogni comportamento dell'anatra
  - L'ereditarietà non è sufficiente, proviamo con le interfacce…




- E' una buona scelta?

# Codificare i comportamenti mediante interfacce

- Una buona scelta: permette a ciascuna sottoclasse di implementare i comportamenti che effettivamente essa deve modellare
- Una cattiva scelta: distrugge ogni possibile **RIUSO** del codice, perché – se non riusciamo a definire un metodo di default - dobbiamo reimplementare le interfacce per ogni sottoclasse



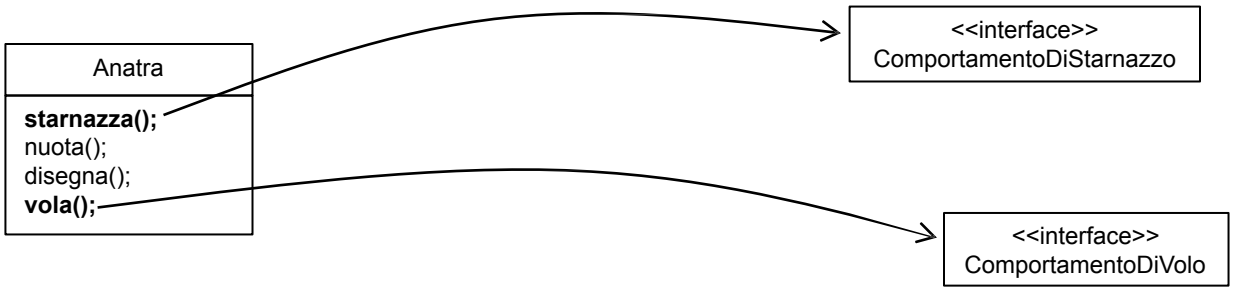
**Un vero incubo!**

# Abbiamo bisogno di un Design Pattern!

**Principio di design:** identifica gli aspetti della tua applicazione che variano e separali da quelli che rimangono uguali

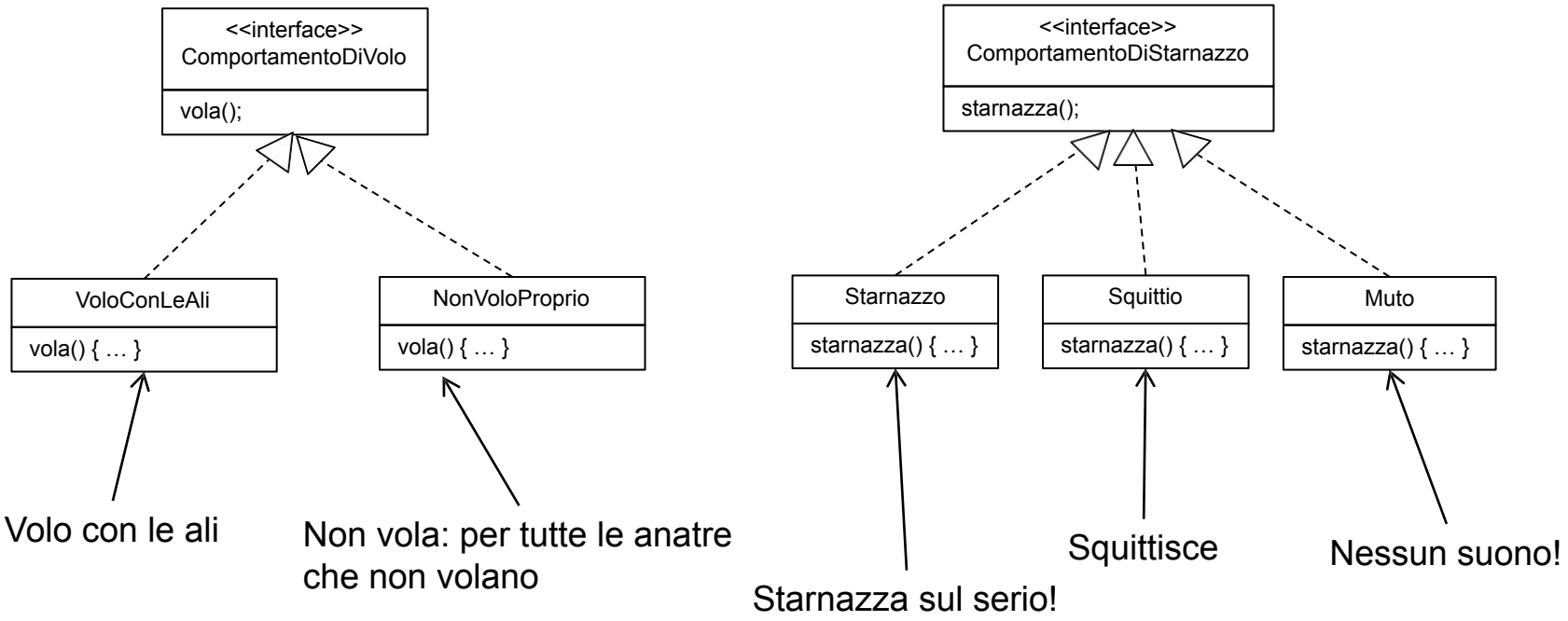


- Come separarli? **Incapsulandoli**
- In questo modo tali parti del sistema varieranno in modo indipendente dalle altre parti
- Eliminiamo i metodi corrispondenti ai comportamenti che vogliamo **modellare a parte**:



# Modellare i comportamenti di una classe

- Progettiamo un'interfaccia funzionale per ogni comportamento
- Per ogni possibile tipo di comportamento implementiamo l'interfaccia



# Integrare i comportamenti nella classe

- La classe Anatra ora **delega** i suoi comportamenti di volo e starnazzo, invece di implementarli direttamente

**PRIMA**

| Anatra |
| --- |
| **starnazza();**<br>nuota();<br>disegna();<br>**vola();** |

**DOPO**

| Anatra |
| --- |
| **ComportamentoDiVolo compVolo;**<br>**ComportamentoDiStarnazzo compStarnazzo;** |
| nuota();<br>disegna();<br>**volo() { compVolo.vola(); }**<br>**starnazzo() { compStarnazzo.starnazza(); }** |

- Dove impostare il comportamento di volo specifico per ciascuna sottoclasse di Anatra?
- Nel costruttore di ciascuna sottoclasse:

```
public class AnatraDomestica extends Anatra
{
   public AnatraDomestica() { compVolo = new VoloConAli(); compStarnazzo = new Starnazzo(); }
}
```

# Metodi “personalizzabili”

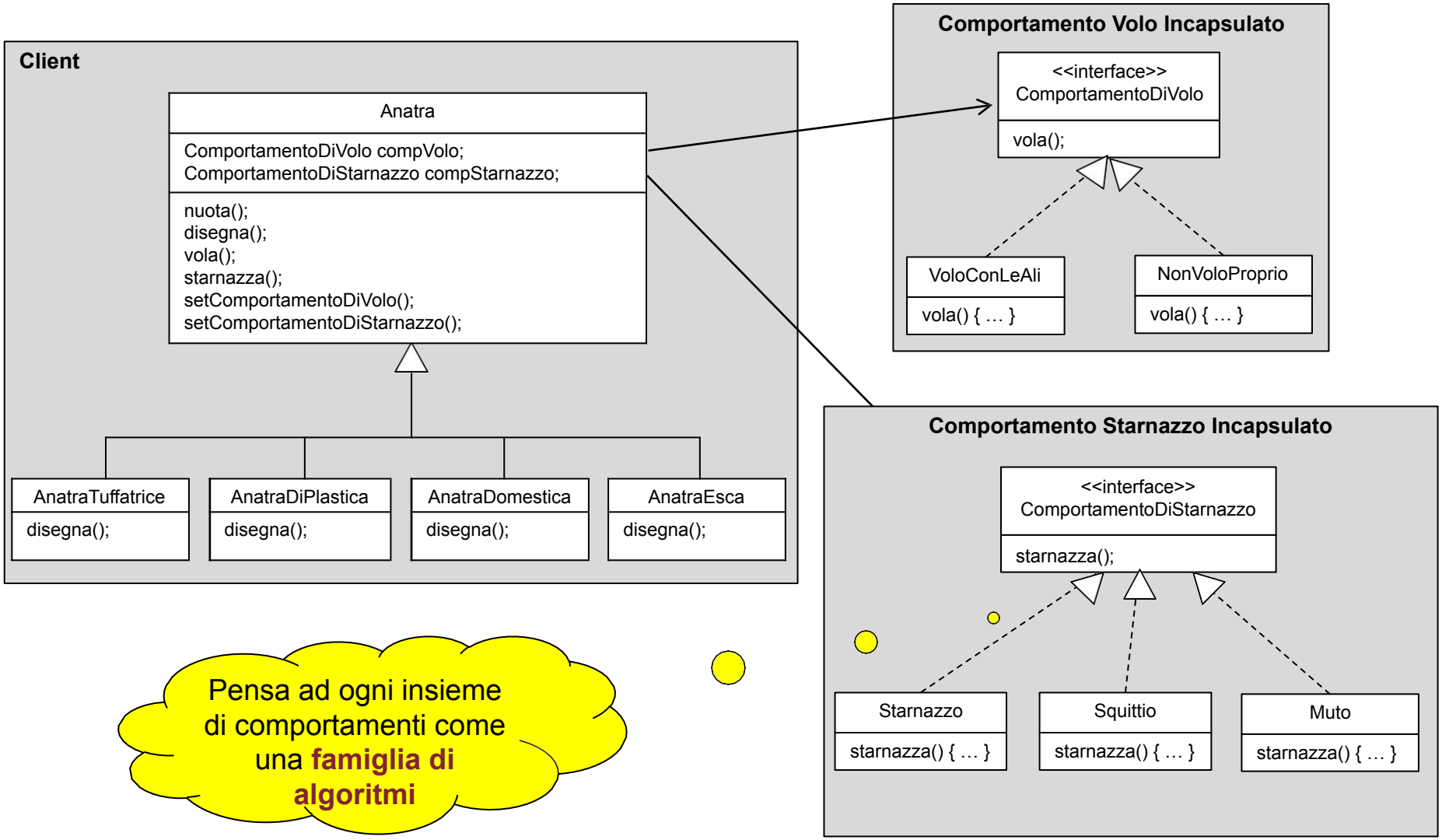
- Grazie all’incapsulamento separato dei comportamenti, abbiamo in realtà dei metodi “personalizzabili”
- Ad esempio, potremmo aggiungere alla classe Anatra i metodi:

```
protected void setComportamentoDiVolo(ComportamentoDiVolo c)
{
    compVolo = c;
}
protected void setComportamentoDiStarnazzo(ComportamentoDiStarnazzo c)
{
    compStarnazzo = c;
}
```

# □□TPS

- Dove abbiamo già visto questo pattern?

# Strategy Pattern: The Big Picture



# Strategy pattern in Java 8!

- Come realizzare lo strategy pattern in Java 8?
- Le varie implementazioni di un metodo possono essere fornite direttamente in input come espressioni lambda o riferimenti ai metodi!
- Es.




```
ComportamentoDiStarnazzo st;
…
st = Starnazzo::starnazza;
st = Squittio::starnazza;
st = Muto::starnazza;
```







# Che principio abbiamo utilizzato?

**Principio di design:**

preferisci la composizione all'ereditarietà



- Invece di ereditare il comportamento, le anatre ottengono il loro comportamento mediante una composizione di oggetti di comportamento
- Implementa il principio della delega (delegation), spostando la responsabilità sulla classe delegata

# Congratulations on your first Strategy Design Pattern





# Esercizio: String2Integer

- Modificare il seguente codice della classe String2Integer in modo da rendere dinamico il comportamento di trasformazione di una stringa in intero:

```
public class String2Integer
{
    private String s;

    public String2Integer(String s) { this.s = s; }

    public int toParsedInteger() { return Integer.parseInt(s); }
    public int toIntegerSize() { return s.length(); }
    public int toIntegerCharSum() { return s.chars().reduce(0, (x, y) -> x+y); }
}
```



# Soluzione: String2Integer

```
public class String2Integer
{
    private String s;
    // o, in alternativa, ToIntFunction (che restituisce un int)
    private Function<String, Integer> method = PARSE_INT;

    final public static Function<String, Integer>
            PARSE_INT = Integer::parseInt,
            STRING_LENGTH = String::length,
            CHAR_SUM = s -> s.chars().reduce(0, (x, y) -> x+y);

    public String2Integer(String s) { this.s = s; }

    public void setToInt(Function<String, Integer> method)
    {
        this.method = method;
    }

    public int toInt() { return method.apply(s); }
}
```

```
public static void main(String[] args)
{
    String2Integer si = new String2Integer("423");
    si.setToInt(STRING_LENGTH);
    System.out.println(si.toInt());
    si.setToInt(PARSE_INT);
    System.out.println(si.toInt());
    si.setToInt(CHAR_SUM);
    System.out.println(si.toInt());
}
```

```
3
423
153
```



# Esercizio: Array comparabile in modo flessibile

- Si progetti una classe generica che modella un array di elementi
  - La classe fornisce un metodo di comparazione standard in ordine naturale del tipo generico
- La classe, mediante lo strategy pattern, permette in fase di costruzione di modificare il modo di comparare gli array:
  - In ordine lessicografico (dal minore al maggiore)
  - In ordine lessicografico inverso (dal maggiore al minore)
  - Per dimensione degli array
- Utilizzare, laddove possibile, Comparator.comparing() e .reversed()



# FilteredFile

- Reingegnerizzare la seguente classe in modo da poter aggiungere con un metodo addFilter una sequenza qualsiasi di filtri a scelta, pur **mantenendo il calcolo del filtro sul file nel metodo** stream() e su una singola riga

```
public class FilteredFile
{
    private Path path;

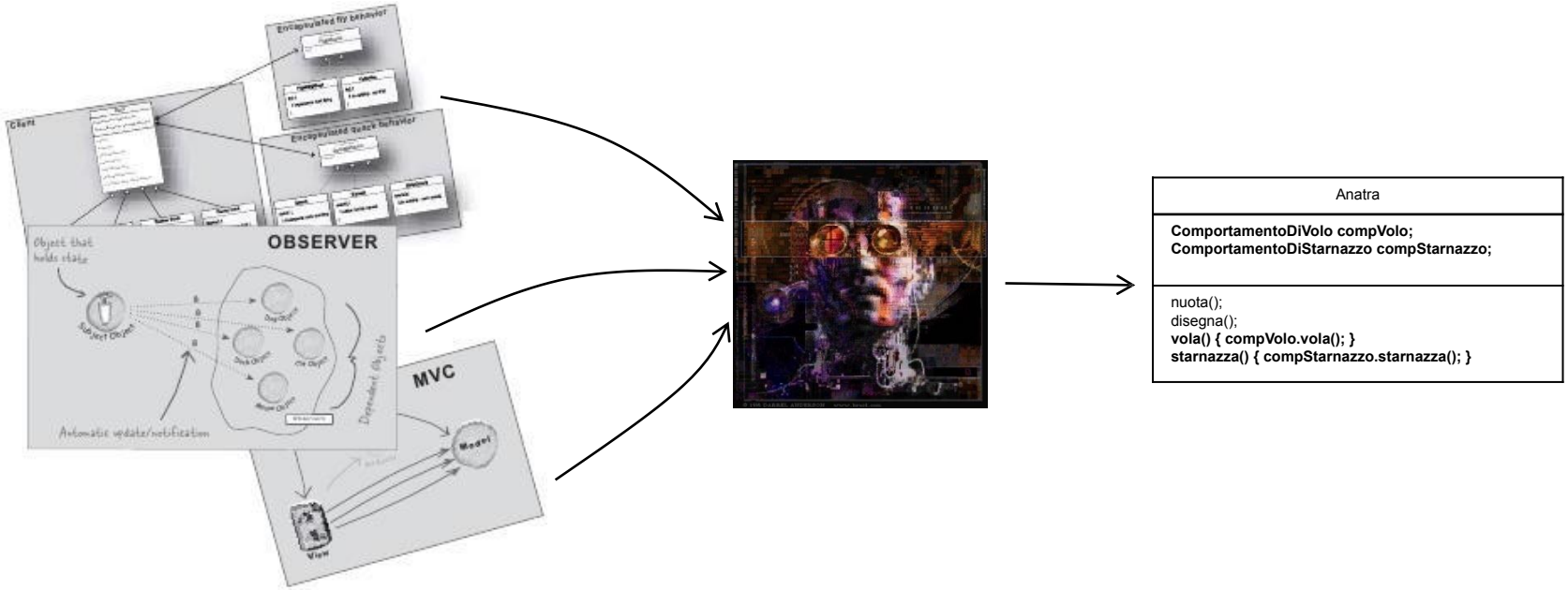
    private boolean filter1(String s) { return s.startsWith(" "); }
    private boolean filter2(String s) { return s.contains(" "); }

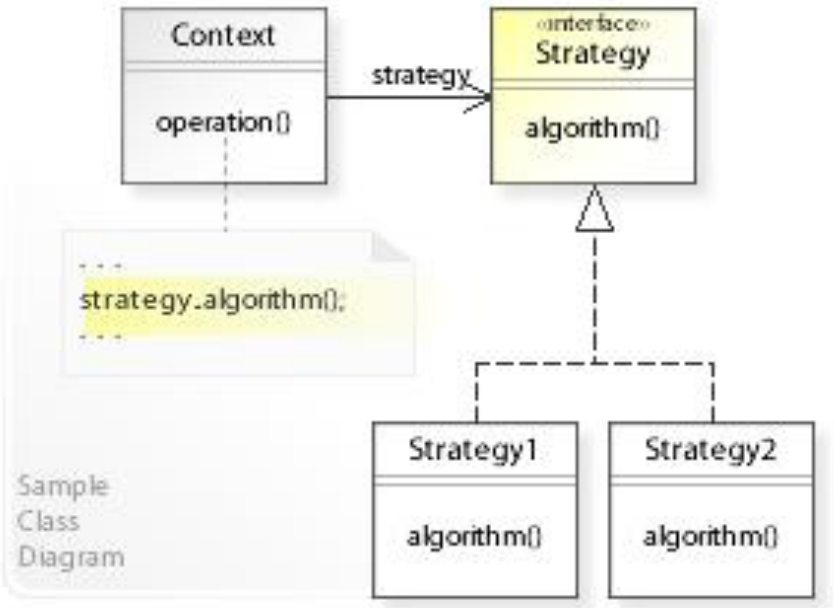
    public FilteredFile(Path path)
    {
        this.path = path;
    }

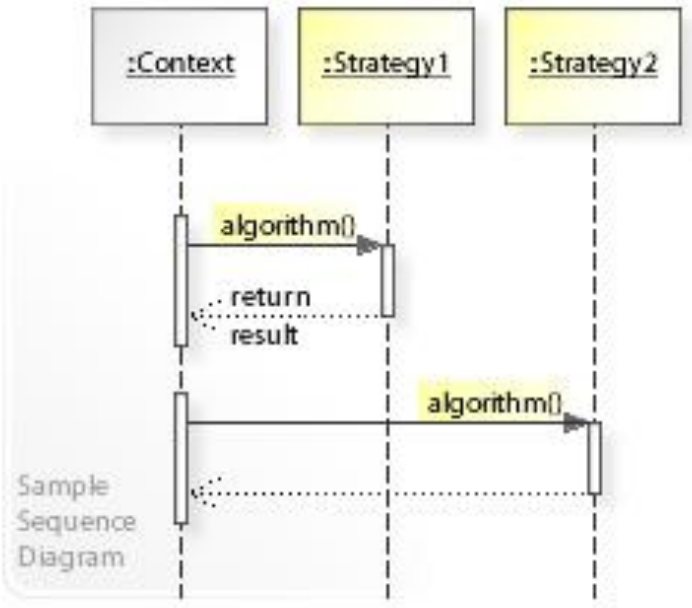
    public Stream<String> stream() throws IOException
    {
        return Files.lines(path).filter(this::filter1).filter(this::filter2);
    }
}
```

# Cos'è un Design Pattern

- E' un modo di **PENSARE** il codice e la struttura a oggetti
- I pattern sono il "succo" dei principi di progettazione orientata agli oggetti
- I Design Pattern non vanno direttamente nel codice
- Vanno **prima** nella tua mente:




Context
operation()
strategy
«interface»
Strategy
algorithm()
. . .
strategy.algorithm();
. . .
Strategy1
algorithm()
Strategy2
algorithm()
Sample
Class
Diagram


:Context
:Strategy1
:Strategy2
algorithm()
return
result
algorithm()
Sample
Sequence
Diagram

# □□TPS

- Quali sono le motivazioni per Strategy?
- Quando si applica Strategy?

# Intento

Definire una famiglia di algoritmi, incapsulare ciascuno di essi e renderli intercambiabili. Il pattern Strategy permette all'algoritmo di variare indipendentemente dai client che lo utilizzano.

# Problema

- Consideriamo una classe Context che esegue una particolare operazione (e.g., ordinamento).
- Diversi modi per eseguire questa operazione (e.g., quicksort, mergesort o heapsort).
- Codificare tutti questi algoritmi nella classe Context complica introdurre un nuovo algoritmo: o modificando il codice di Context, o imponendo una modifica alle sottoclassi.

# Soluzione

- definire i comportamenti concreti in classi separate
- rendere la classe Context configurabile per utilizzarne uno qualsiasi.
- Context può essere facilmente esteso per utilizzare un nuovo algoritmo di ordinamento senza modificare il suo codice.

# Applicabilità

- Molte classi correlate differiscono solo nel loro comportamento.
- Hai bisogno di varianti diverse di un algoritmo.
- Un algoritmo utilizza dati che i client non dovrebbero conoscere.
- Una classe definisce molti comportamenti, e questi appaiono come molte dichiarazioni condizionali nelle sue operazioni. Invece di molti condizionali, sposta i rami condizionali correlati nella loro propria classe Strategy.

# Alcuni esempi di uso del pattern Strategy

- Ordinamento (vogliamo ordinare i numeri, ma non sappiamo a priori quale algoritmo utilizzare)
- Giochi (composizionalità dei comportamenti degli oggetti: ad esempio, un giocatore inizialmente può solo camminare e saltare, ma più avanti potrà eseguire altre azioni più complesse: nuotare, teletrasportarsi ecc.; astronavi con potenze di fuoco variabili)
- Produrre output: vogliamo produrre in output una stringa che rappresenti l'oggetto, ma anche un XML, in JSON, un CSV, ecc.
- Validazione: vogliamo verificare i nostri elementi secondo certe regole, ma potremmo voler aggiungere ulteriori regole più avanti

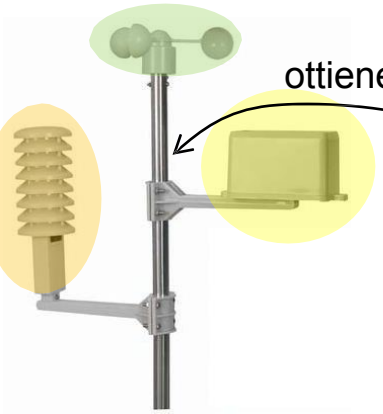
# Un secondo caso concreto: monitoraggio del tempo

- Supponiamo di dover costruire una stazione di monitoraggio del tempo basata su Internet
- Basata sul nostro tipo TempoAggiornato che tiene traccia delle condizioni di tempo
  - Temperatura
  - Umidità
  - Pressione
- Vogliamo un'applicazione che visualizzi in tempo reale:
  1. Le attuali condizioni
  2. Statistiche sul tempo
  3. Una semplice previsione del tempo
- Vogliamo rilasciare un'API per permettere ad altri sviluppatori di scrivere i loro programmi basati sui nostri dati (pagando per ogni chiamata all'API!)

# Schema dell'applicazione di monitoraggio del tempo

Dati forniti dalla stazione del tempo

Ciò che **NOI** implementiamo



ottiene dati da

TempoAggiornato

visualizza




stazione del tempo

dispositivo con display

# Come realizzare la classe TempoAggiornato?

- Implementiamo 4 metodi:

| TempoAggiornato |
| --- |
| getTemperatura();<br>getUmidità();<br>getPressione();<br>misurazioniCambiate(); |

- I primi tre metodi ("**getter**") restituiscono le misurazioni più recenti per temperatura, umidità e pressione
- La classe sa come aggiornarle
- Ogni volta che vengono aggiornate le misurazioni viene chiamato il metodo misurazioniCambiate
  - Così possiamo aggiornare lo schermo del visualizzatore



# Una prima implementazione di TempoAggiornato

```
public class TempoAggiornato
{
    Display displayCondizioniAttuali;
    Display displayStatistiche;
    Display displayPrevisioni;

    public void misurazioniCambiate()
    {
        double temp = getTemperatura();
        double umidita = getUmidita();
        double press = getPressione();

        displayCondizioniAttuali.aggiorna(temp, umidita, press);
        displayStatistiche.aggiorna(temp, umidita, press);
        displayPrevisioni.aggiorna(temp, umidita, press);
    }

    // altri metodi
    // ...
}
```

ottiene le misurazioni più recenti

aggiorna i tre display

Tuttavia l'interfaccia sembra uguale per tutti i display!

Stiamo codificando implementazioni **CONCRETE**
(nessun modo di aggiungerne altre senza cambiare il programma)

## 👐TPS

- Come possiamo rendere questo meccanismo flessibile in modo da non dover “cablare” l’elenco dei dispositivi che vogliono ricevere le informazioni aggiornate?
- Quale pattern già visto si può applicare?

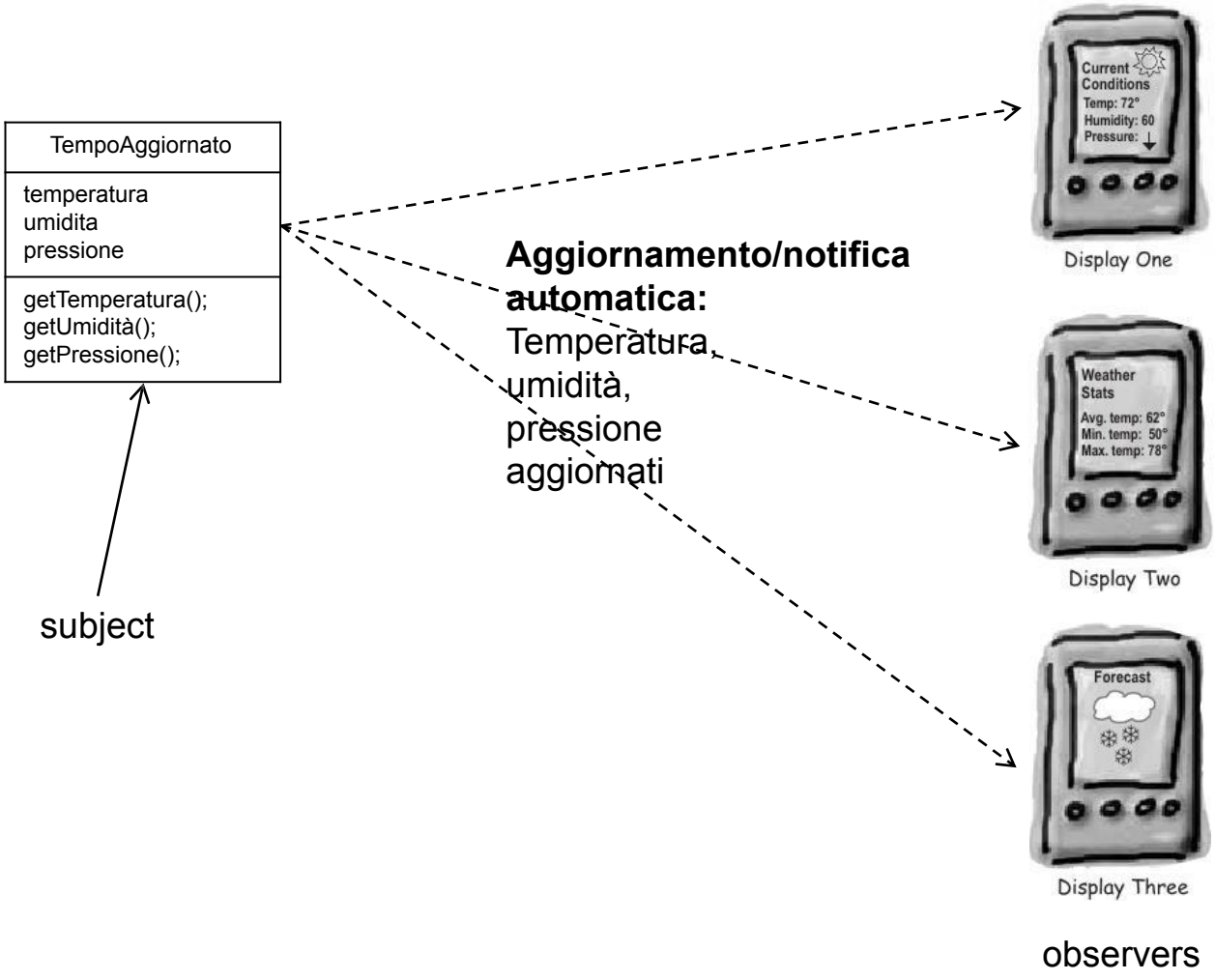
# Possiamo utilizzare l'Observer Pattern



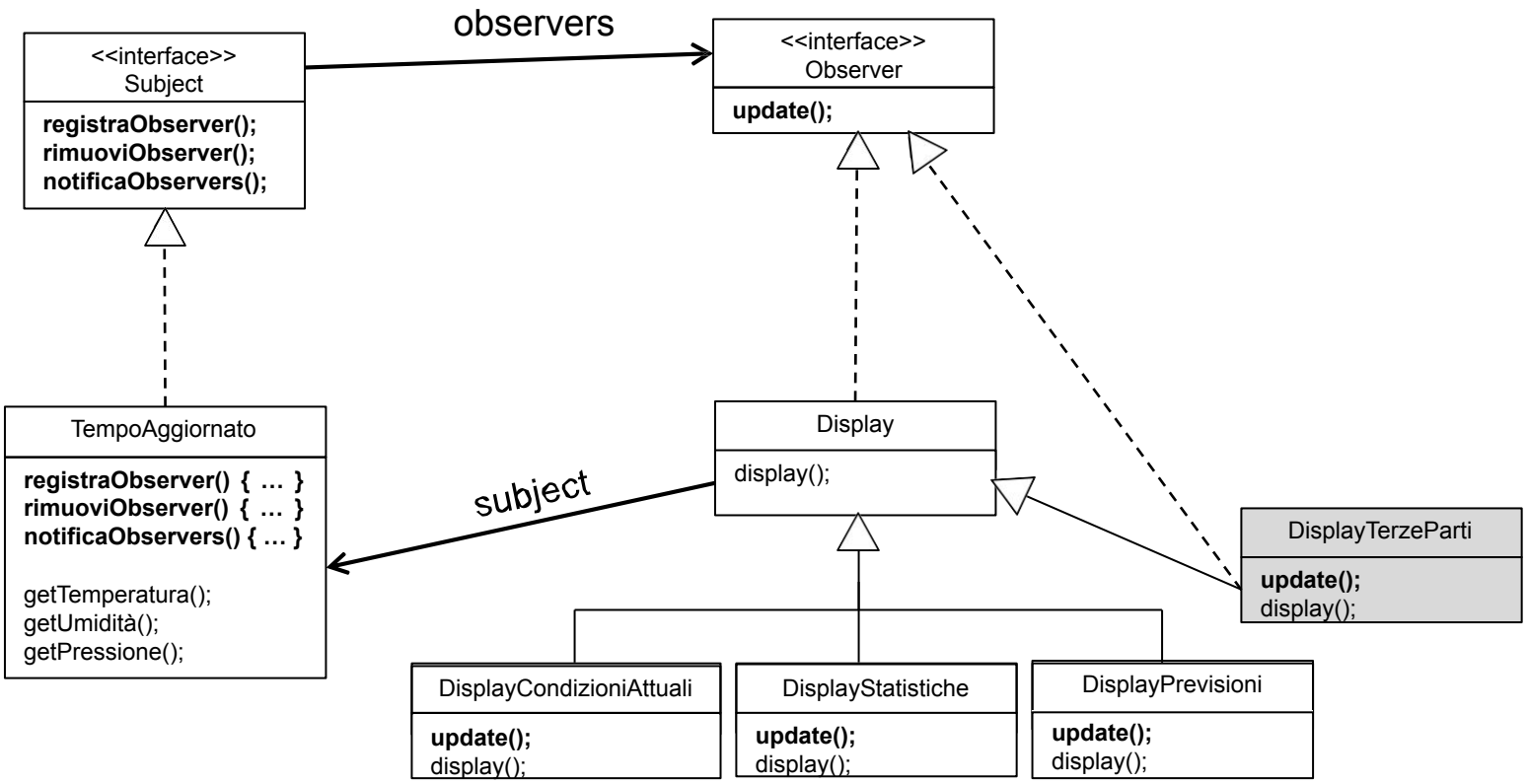

Come l'abbonamento a un settimanale:

1. Panini stampa Topolino
2. Ti abboni a un **determinato** settimanale e, ogni volta che viene stampata una nuova edizione, ti viene consegnata
3. Cancelli l'abbonamento quando non vuoi più ricevere le nuove copie
4. L'editore rimane nel business: altre persone continuano ad abbonarsi e a cancellare l'abbonamento

# Subject + Observer = Observer Pattern



# Subject (o Observable) + Observer = Observer Pattern



# Sporchiamoci le mani: le interfacce

```
public interface Subject
{
    void registraObserver(Observer o);
    void rimuoviObserver(Observer o);
    void notificaObservers();
}

public interface Observer
{
    void update(double temp, double umidita, double press);
}
```

# Sporchiamoci le mani: le implementazioni (1)

```
public class TempoAggiornato implements Subject
{
    private double temperatura;
    private double umidita;
    private double pressione;
    private ArrayList<Observer> observers = new ArrayList<Observer>();

    public void misurazioniCambiate()
    {
        notificaObservers();
    }

    @Override
    public void registraObserver(Observer o)
    {
        observers.add(o);
    }

    @Override
    public void rimuoviObserver(Observer o)
    {
        observers.remove(o);
    }

    @Override
    public void notificaObservers()
    {
        for (Observer o : observers) o.update(temperatura, umidita, pressione);
    }

    // altri metodi qui
}
```

# Sporchiamoci le mani: le implementazioni (2)

```
abstract public class Display implements Observer
{
    abstract public void display();
}

public class DisplayCondizioniAttuali extends Display
{
    private double temperatura;
    private double umidita;
    private double pressione;
    private Subject subject;

    public DisplayCondizioniAttuali(Subject subject)
    {
        this.subject = subject;
        subject.registraObserver(this);
    }

    public void display()
    {
        System.out.println("Condizioni attuali: temp="+temperatura
                +", umidità="+umidita+", pressione = "+pressione);
    }

    public void update(double temp, double umidita, double press)
    {
        this.temperatura = temp;
        this.umidita = umidita;
        this.pressione = press;
        display();
    }
}
```

# Utilizzo principale del pattern Observer

- GUI e gestione degli eventi
- Un oggetto si "registra" per ricevere ogni input di un certo tipo da un elemento della GUI
- Ad esempio:
  - Un bottone "grafico" pubblica gli eventi che si verificano su di esso (es. bottone cliccato)
  - Un Listener implementa un observer
  - Si registra il Listener (in ascolto) sull'elemento della GUI
  - Ecc.

# Che principio abbiamo utilizzato?



**Principio di design:**

progetta in modo da accoppiare gli oggetti che interagiscono tra loro



- L'Observer Pattern permette di comunicare uno stato o un evento a un insieme di oggetti nel modo il più possibile svincolato



# Esercizio: Borsa (1)

- Si considerino le seguenti cinque classi:

```
public class Borsa
{
    private Analizzatore analizzatoreStatistiche;
    private Analizzatore analizzatoreTrend;

    // chiamato ogni volta che una nuova transazione è disponibile
    public void nuovaTransazione(Transazione t)
    {
        analizzatoreStatistiche.analizza(t);
        analizzatoreTrend.analizza(t);
    }
}
```

```
abstract public class Analizzatore
{
    abstract public void analizza(Transazione t);
}
```

```
public class AnalizzatoreStatistiche extends Analizzatore
{
    private List<Transazione> transazioni = new ArrayList<Transazione>();

    @Override
    public void analizza(Transazione t)
    {
        transazioni.add(t);
    }

    @Override
    public String toString()
    {
        double importo = 0.0;
        for (Transazione t : transazioni) importo += t.getImporto();
        return "IMPORTO MEDIO DELLE TRANSAZIONI = "+importo/transazioni.size();
    }
}
```



# Esercizio: Borsa (2)

```java
public class AnalizzatoreTrend extends Analizzatore
{
    private Transazione ultimaTransazione;
    private double trendImporto;

    @Override
    public void analizza(Transazione t)
    {
        if (ultimaTransazione == null) trendImporto = t.getImporto();
        else trendImporto = t.getImporto()-ultimaTransazione.getImporto();

        ultimaTransazione = t;
    }

    @Override
    public String toString()
    {
        return "TREND = "+trendImporto;
    }
}
```

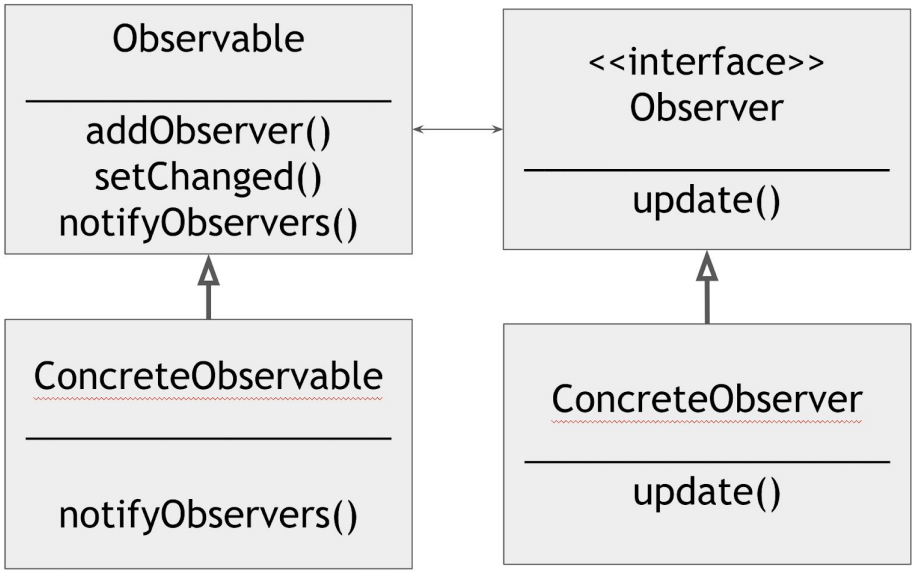
```java
public class Transazione
{
    private String compagnia;
    private double importo;

    public Transazione(String compagnia, double importo)
    {
        this.compagnia = compagnia;
        this.importo = importo;
    }

    public String getCompagnia() { return compagnia; }
    public double getImporto() { return importo; }
}
```

- Si mostrino le criticità e si ristrutturi il codice utilizzando il pattern Observer

# ObserverObservable Pattern (in java)




**Attivazione**

```
ConcreteObservable observable=new ConcreteObservable();
ConcreteObserver observer=new ConcreteObserver();
observable.addObserver(observer);
```

**Invio di un Messaggio**

(all'interno di un metodo di observable

```
.....
setChanged();
notifyObserver(oggetto);

.....
```

**Ricezione di un Messaggio**

```
update(Observable observable, Object oggetto)
{
  ....
  ...
}
```

**Mittente**

**Destinatario**

Comunicazione attivata a runtime / disaccoppiamento

**L'Observer Pattern permette di comunicare uno stato o un evento a un insieme di oggetti in modo il più possibile svincolato**

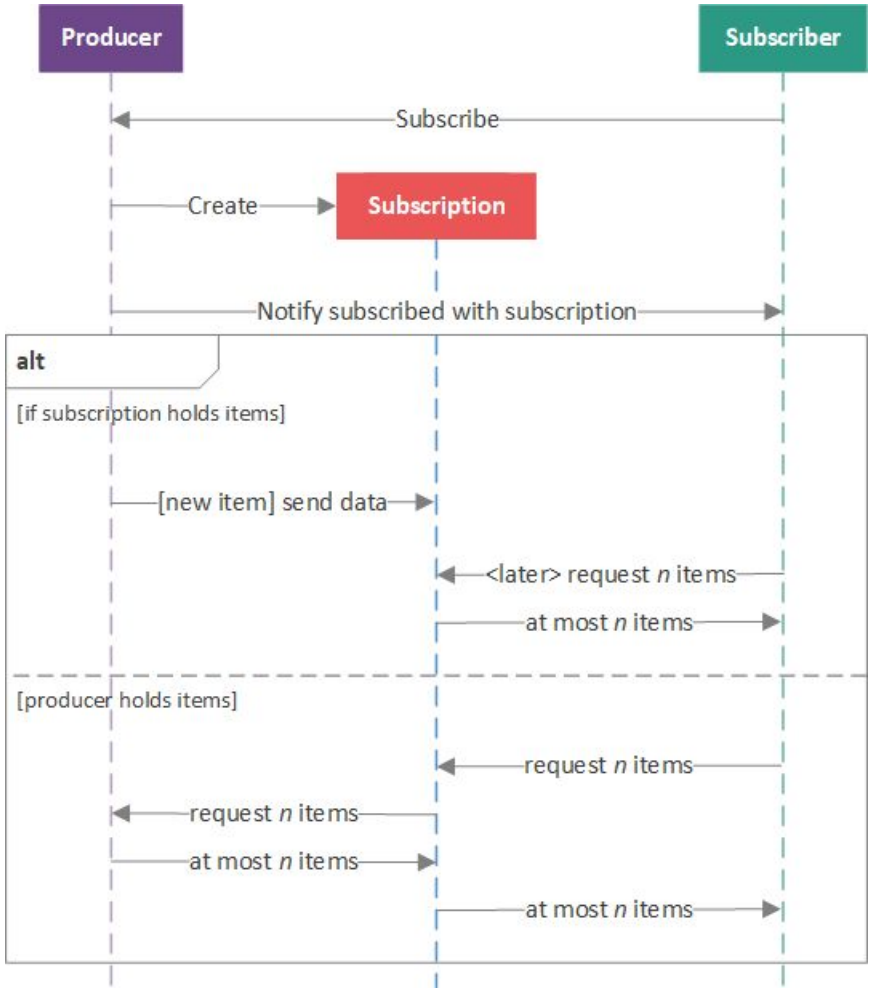


# Reactive stream

- Un paradigma di programmazione che gestisce il **recupero di dati da sorgenti multiple**, (quasi) in **tempo reale**, con **continui invii di informazione**
- L'invio di informazione è tipicamente asincrono
- Non sappiamo in anticipo quando i dati saranno inviati



# Sono coinvolti 4 attori: Producer, Subscriber, Subscription e Processor



# Se un'entità è sia un Producer che un Subscriber, è un Processor

- Un intermediario nello stream di dati
- Permette di concatenare più elaborazioni dei dati in una sorta di **pipeline**



# Flow e le 4 interfacce in Java 9

- Quattro interfacce definite all'interno della classe java.util.concurrent.Flow

***Nested Class Summary***

**Nested Classes**

| Modifier and Type | Class | Description |
|---|---|---|
| static interface | `Flow.Processor<T,R>` | A component that acts as both a Subscriber and Publisher. |
| static interface | `Flow.Publisher<T>` | A producer of items (and related control messages) received by Subscribers. |
| static interface | `Flow.Subscriber<T>` | A receiver of messages. |
| static interface | `Flow.Subscription` | Message control linking a `Flow.Publisher` and `Flow.Subscriber`. |

- Flow.Publisher ha un solo metodo:

***Method Summary***

**All Methods** | **Instance Methods** | **Abstract Methods**

| Modifier and Type | Method | Description |
|---|---|---|
| void | `subscribe(Flow.Subscriber<? super T> subscriber)` | Adds the given Subscriber if possible. |

# Flow e le 4 interfacce in Java 9

- Flow.Subscriber<T> ha 4 metodi:

All Methods | Instance Methods | Abstract Methods

| Modifier and Type | Method | Description |
|---|---|---|
| void | `onComplete()` | Method invoked when it is known that no additional Subscriber method invocations will occur for a Subscription that is not already terminated by error, after which no other Subscriber methods are invoked by the Subscription. |
| void | `onError(Throwable throwable)` | Method invoked upon an unrecoverable error encountered by a Publisher or Subscription, after which no other Subscriber methods are invoked by the Subscription. |
| void | `onNext(T item)` | Method invoked with a Subscription's next item. |
| void | `onSubscribe (Flow.Subscription subscription)` | Method invoked prior to invoking any other Subscriber methods for the given Subscription. |

- Flow.Processor<T,R> estende Subscriber<T> e Producer<R>
- Flow.Subscription ne ha 2:

All Methods | Instance Methods | Abstract Methods

| Modifier and Type | Method | Description |
|---|---|---|
| void | `cancel()` | Causes the Subscriber to (eventually) stop receiving messages. |
| void | `request(long n)` | Adds the given number n of items to the current unfulfilled demand for this subscription. |

# Creare oggetti

- Finora siamo stati abituati a creare oggetti mediante la parola chiave new

```
Anatra anatra = new AnatraDiPlastica();
```



Codice flessibile grazie al polimorfismo

Istanza di una classe concreta

- Tuttavia se fino a tempo di esecuzione **non sappiamo** quale anatra istanziare, dobbiamo scrivere qualcosa del genere:

```
if (diPlastica) anatra = new AnatraDiPlastica();
else if (siCaccia) anatra = new AnatraEsca();
else if (aCasa) anatra = new AnatraDomestica();
```

# Ricorda: separa ciò che cambia da ciò che rimane uguale

**Principio di design:** identifica gli aspetti della tua applicazione che variano e separali da quelli che rimangono uguali



- In questa situazione ciò che cambia è la classe concreta da istanziare

# La classe Pizzeria



```
public class Pizzeria
{
    public Pizza ordinaPizza(String nome)
    {
        Pizza pizza = null;

        // istanzia la classe sulla base del tipo concreto
        // passato in input come stringa
        if (nome.equalsIgnoreCase("margherita"))
            pizza = new PizzaMargherita();
        else if (nome.equalsIgnoreCase("capricciosa"))
            pizza = new PizzaCapricciosa();
        else if (nome.equalsIgnoreCase("funghi"))
            pizza = new PizzaFunghi();
        else if (nome.equalsIgnoreCase("vegetariana"))
            pizza = new PizzaVegetariana();

        // queste operazioni sono uguali per tutti:
        // ma ogni pizza sa come "prepararsi"
        pizza.impasta();
        pizza.cuoci();
        pizza.taglia();
        pizza.inscatola();

        return pizza;
    }
}
```

Ma se voglio aggiungere un'altra pizza? Devo aggiornare il codice!
**MALE, MOLTO MALE!**

# Dobbiamo incapsulare la creazione degli oggetti

- Mediante un nuovo oggetto che chiamiamo Factory
- Factory si occupa della creazione di oggetti di un certo tipo:

```java
public class PizzaFactory
{
    public Pizza creaPizza(String nome)
    {
        Pizza pizza = null;

        // istanzia la classe sulla base del tipo concreto
        // passato in input come stringa
        if (nome.equalsIgnoreCase("margherita"))
            pizza = new PizzaMargherita();
        else if (nome.equalsIgnoreCase("capricciosa"))
            pizza = new PizzaCapricciosa();
        else if (nome.equalsIgnoreCase("funghi"))
            pizza = new PizzaFunghi();
        else if (nome.equalsIgnoreCase("vegetariana"))
            pizza = new PizzaVegetariana();

        return pizza;
    }
}
```

Migliorabile con la Reflection, vero? :-)

```java
public class Pizzeria
{
    private PizzaFactory pizzaFactory;

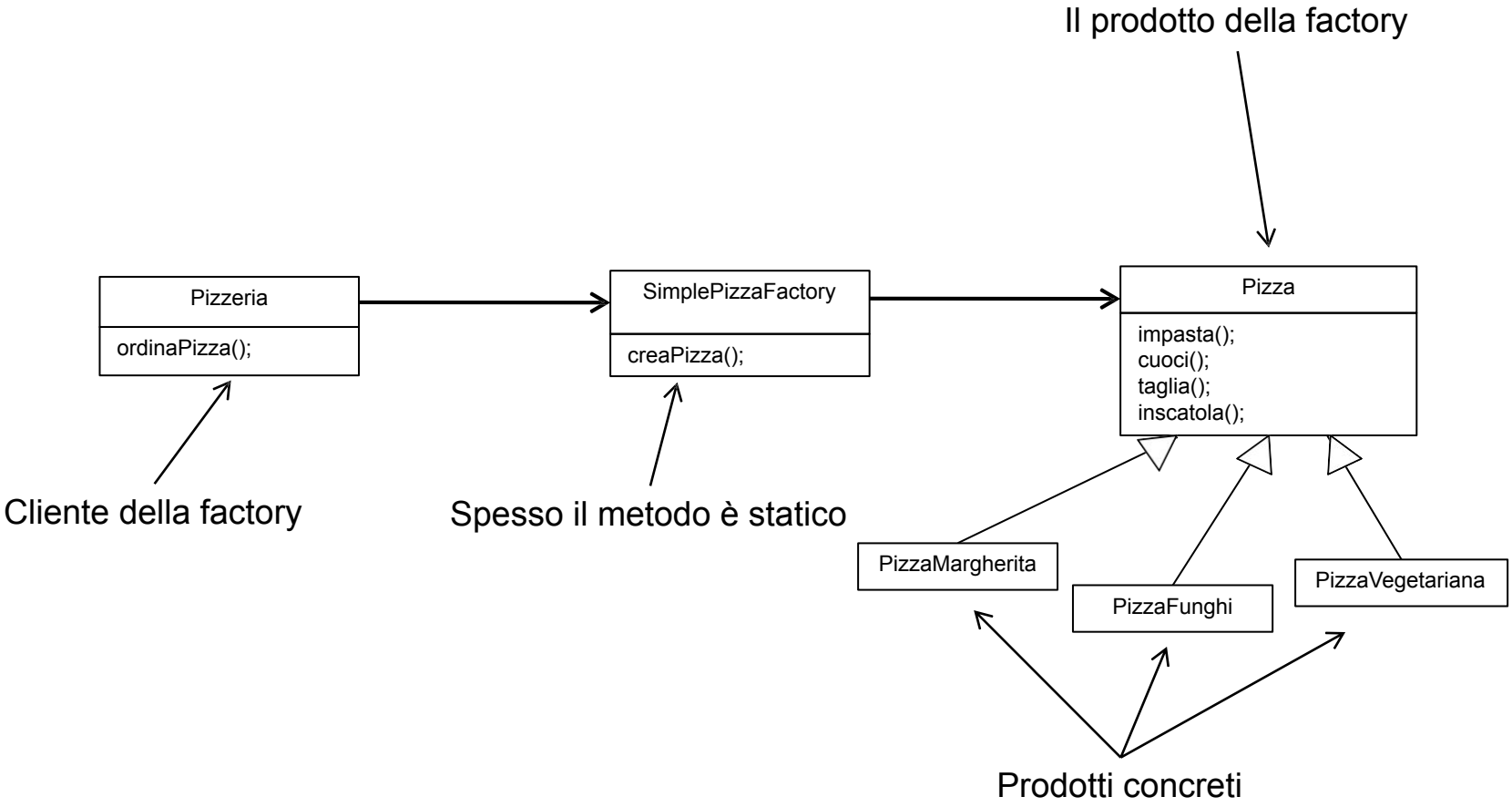
    public Pizzeria(PizzaFactory pizzaFactory)
    {
        this.pizzaFactory = pizzaFactory;
    }

    public Pizza ordinaPizza(String nome)
    {
        Pizza pizza = pizzaFactory.creaPizza(nome);

        // queste operazioni sono uguali per tutti:
        // ma ogni pizza sa come "prepararsi"
        pizza.impasta();
        pizza.cuoci();
        pizza.taglia();
        pizza.inscatola();

        return pizza;
    }
}
```

# Il Simple Factory Pattern



# Esempi di Factory in Java

- Charset.forName
- NumberFormat.getInstance

```
NumberFormat f = NumberFormat.getInstance(Locale.UK);
f.setMaximumFractionDigits(3);
System.out.println(f.format(0.52453));
```

- Class.forName
- List.of, Set.of, Map.of, Stream.of

# E se avviassimo un “franchising” della nostra Pizzeria?

- Le diverse pizzerie si troveranno in posti diversi
- Ognuna produrrà la propria pizza
  - TrasteverePizzaFactory
  - OstiaPizzaFactory
  - TestaccioPizzaFactory
- **No problem!** La passiamo in input alla Pizzeria:

```
Pizzeria laMiaPizzeriaDiOstia = new Pizzeria(new OstiaPizzaFactory());
Pizza margherita = laMiaPizzeriaDiOstia.ordinaPizza("margherita");
```

- Tuttavia vorresti **vincolare** una factory a un determinata pizzeria
  - Altrimenti tutti (!) useranno il pizzaiolo di Ostia (il più bravo?)

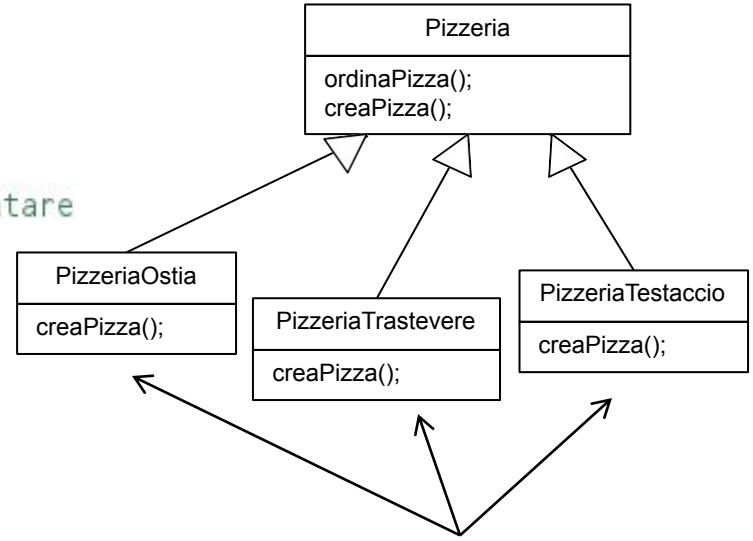
# Un framework generale per la Pizzeria

```
abstract public class Pizzeria
{
    public Pizza ordinaPizza(String nome)
    {
        // chiama creaPizza: metodo astratto da implementare
        Pizza pizza = creaPizza(nome);

        // queste operazioni sono uguali per tutti:
        // ma ogni pizza sa come "prepararsi"
        pizza.impasta();
        pizza.cuoci();
        pizza.taglia();
        pizza.inscatola();

        return pizza;
    }

    // ogni pizzeria "concreta" sara' costretta a implementare creaPizza
    // "a modo suo" (pizza più bruciacchiata, pizza più alta, ecc.)
    abstract protected Pizza creaPizza(String nome);
}
```

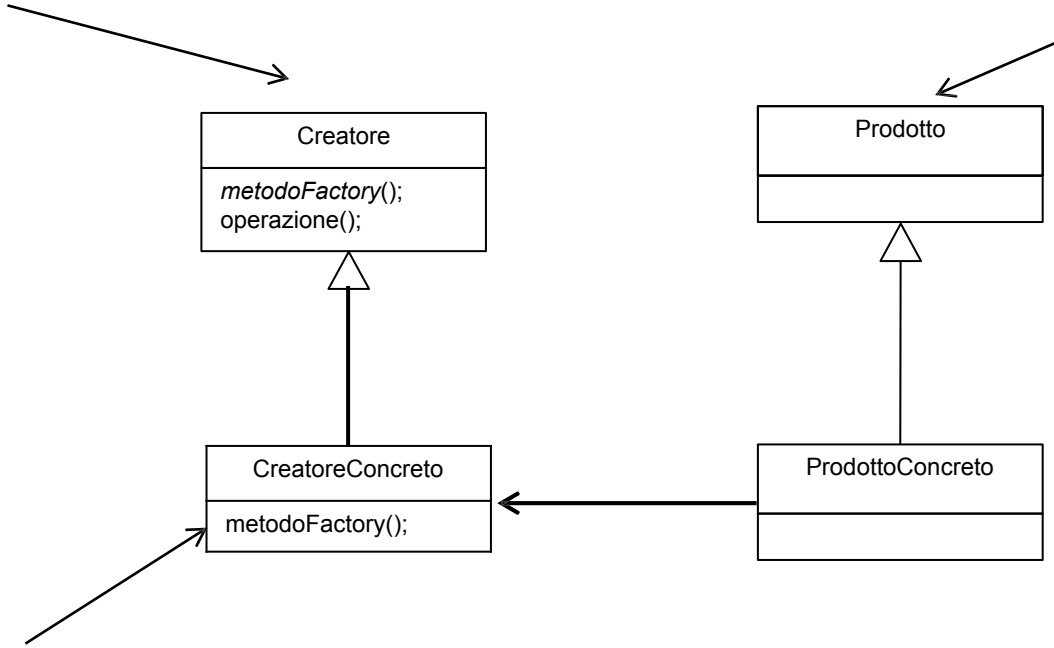



La pizzeria concreta è diventata la specifica factory

# Il Factory Pattern

La classe che contiene
Le implementazioni per tutti
i metodi che manipola
prodotti, eccetto il metodo
factory

Tutti i prodotti
implementano la stessa
interfaccia disponibile nella
classe astratta




L'unica classe che ha la
conoscenza per costruire i
prodotti concreti

# Riferimento a costruttori esistenti

- E’ possibile passare riferimenti a metodi esistenti
- Utilizzando la sintassi:
  - Classe::new
- Perfetto per il (simple) factory pattern!



# Che principio abbiamo utilizzato?

**Principio di design:**

Nel costruire oggetti, lascia alle sottoclassi la possibilità di decidere quale classe istanziare



- Il Factory Pattern permette a una classe di differire l'istanziazione alle sottoclassi



# Esercizio: Scrittori di libri (1)

- Si considerino le seguenti classi:

```
public class Scrittore
{
    public enum GenereLibro
    {
        AVVENTURA, GIALLO, FUMETTO
    }

    public Libro pubblica(GenereLibro genere)
    {
        Libro libro = null;

        switch(genere)
        {
            case AVVENTURA: libro = new LibroAvventura(); break;
            case GIALLO: libro = new LibroGiallo(); break;
            case FUMETTO: libro = new Fumetto(); break;
        }

        libro.impagina();

        return libro;
    }
}
```

```
public class ScrittoreDiGialli extends Scrittore
{

}
```

```
public class Fumettista extends Scrittore
{

}
```



# Esercizio: Scrittori di libri (2)

```
abstract public class Libro
{
    public void impagina() { /* ... */ }
}

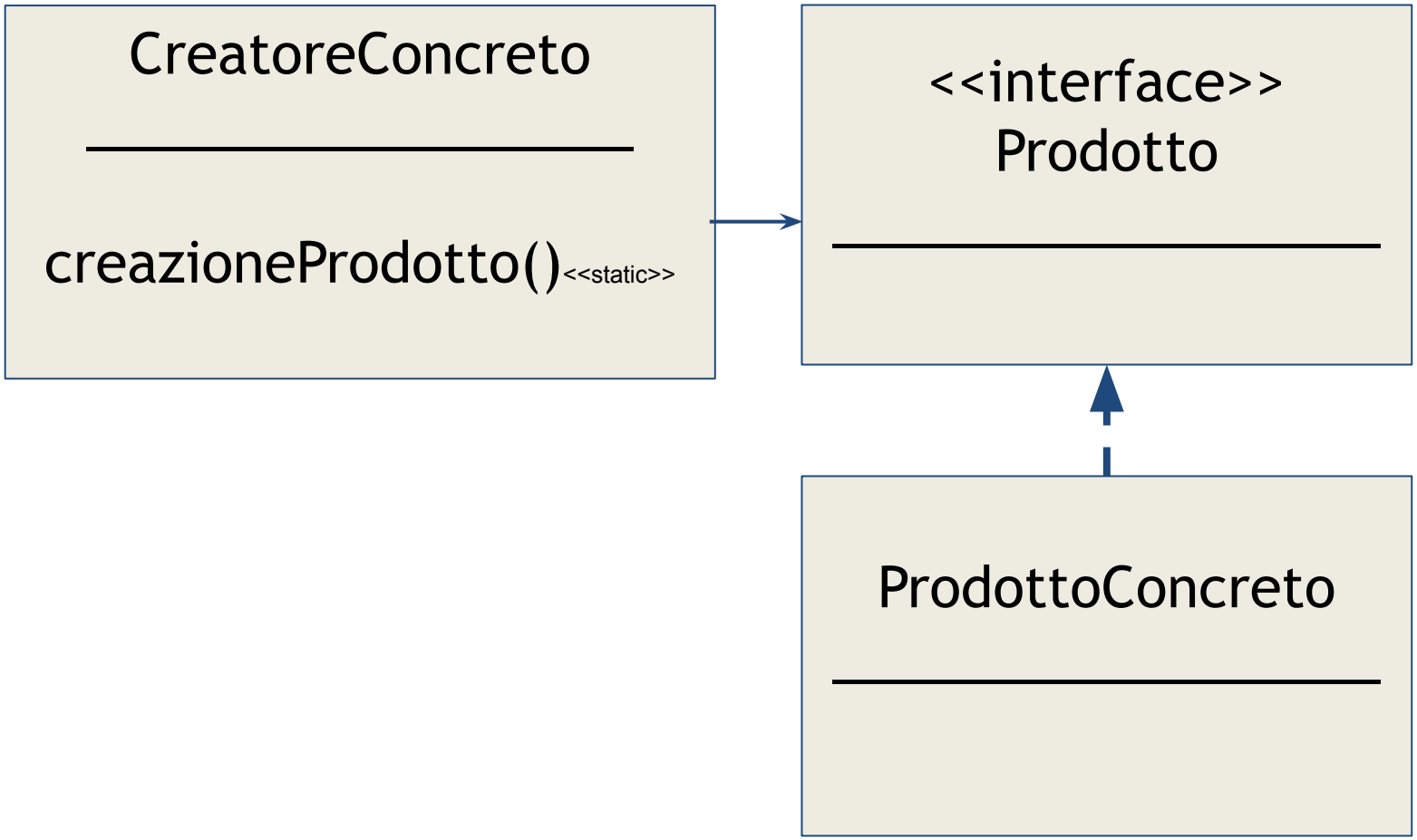
public class LibroGiallo extends Libro
{
}

public class LibroAvventura extends Libro
{
}

public class Fumetto extends Libro
{
}
```

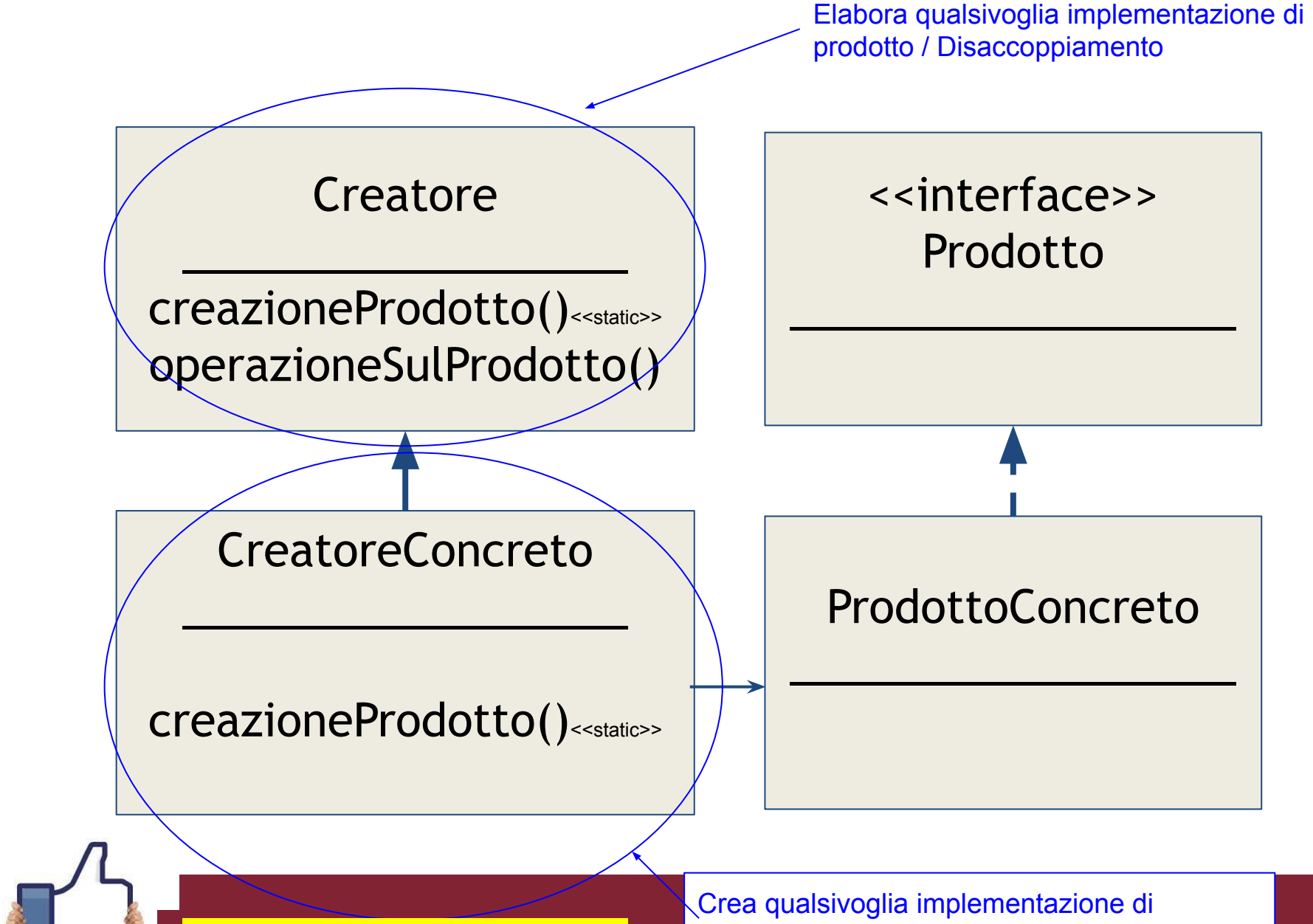
- Si mostrino le criticità e si ristrutturi il codice utilizzando il pattern Factory

# Simple Factory






# Factory Pattern






**Perché ci piace questo pattern?**

Volete creare un'applicazione che aiuti a configurare un nuovo PC. Per esempio, volete costruire 3 pc:

- uno con CPU a, 32GB di RAM, 1TB storage
- uno con CPU a, 32GB di RAM, 1TB storage, ed una scheda grafica dedicata
- uno con CPU a, 32GB di RAM, 1TB storage, ed un modem 5G

- quale difficoltà prevedete nell'implementazione in Java?

```
        Computer gamingRig = new Computer.ComputerBuilder("Intel i9", 32, 1024)
                .setGPU("NVIDIA RTX 3080")
                .setOS("Windows 11")
                .setBluetooth("Bluetooth 5.0")
                .setWiFi("WiFi 6")
                .setStylus("Wacom Stylus")
                .setFingerprintReader("Fingerprint Reader V3")
                .build();
        Computer basicPC = new Computer.ComputerBuilder("Intel i5", 8, 256)
                .build();
        System.out.println(gamingRig);
        System.out.println(basicPC);
```

# Builder pattern

- E' un altro pattern per la creazione di oggetti
- Utile quando si hanno molti parametri, alcuni opzionali
- Creare troppi costruttori rende poco agevole la progettazione
  - Anche delle sottoclassi
- Un esempio concreto: l'interfaccia java.util.stream.Stream
  - I metodi per le operazioni intermedie "configurano" la costruzione dell'oggetto
  - I metodi terminali restituiscono l'oggetto da costruire

# Builder: passi

1. Crea una nuova classe, tipicamente chiamata ClasseBuilder se Classe è la classe di cui si vogliono costruire gli oggetti
   - ClasseBuilder può anche essere una classe statica annidata all'interno di Classe (in quel caso, si chiamerà Classe.Builder)
   - Il costruttore di Classe viene impostato come privato o si usa la visibilità di package se ClasseBuilder non è annidato
2. ClasseBuilder è dotata di metodi per l'impostazione dei valori iniziali dei campi dell'oggetto da costruire
   - Ciascuno di questi metodi restituisce l'istanza del builder (this), in modo che si possano effettuare chiamate in cascata, come in StringBuffer.append
3. ClasseBuilder è dotata di un metodo build che restituisce un'istanza di Classe **(o di una sua sottoclasse!)** secondo le impostazioni specificate
4. ClasseBuilder può essere dotata un costruttore per obbligare l'impostazione iniziale di uno o più campi dell'oggetto da costruire

# Vantaggi e svantaggi

Vantaggi:

- Permette di rendere più flessibile la costruzione di oggetti con molti parametri
- Rende il codice di costruzione più leggibile
- Permette di evitare il passaggio di parametri null o poco chiari
- Permette di evitare di avere stati intermedi non validi dell'oggetto costruito

Svantaggi:

- La costruzione richiede più chiamate di metodi



# Creare libri con il pattern Builder

- Utilizzare il pattern Builder per creare un volume
- I tipi di volume creabili sono:
  - Raccolta rilegata di fogli A4
  - Raccoglitore di volumi
  - Fumetto
  - Romanzo



# Esercizio con il pattern Builder

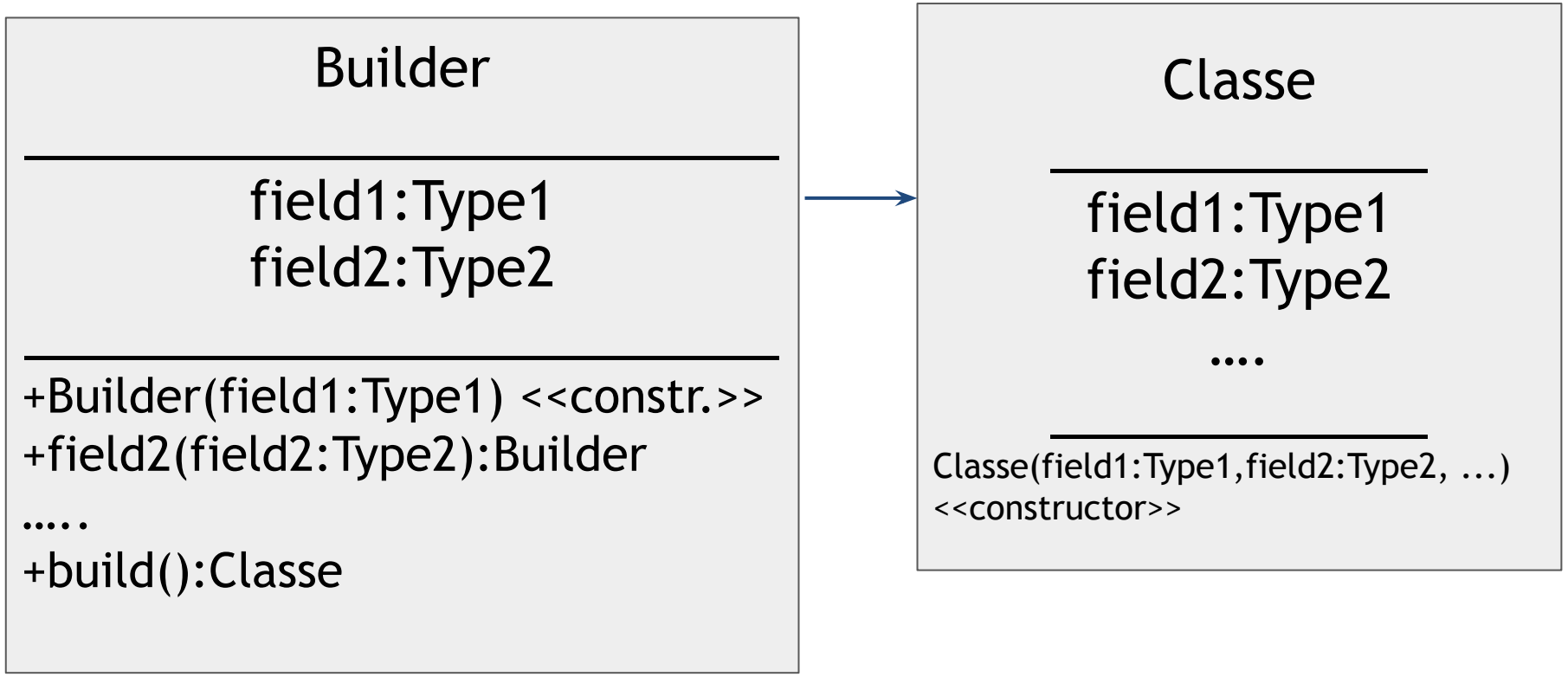
- Utilizzare il pattern Builder per costruire una collezione di interi
- Il Builder dispone dei seguenti comportamenti:
  - add(i1, …, ik) – aggiunge gli interi specificati
  - removeAll(Predicate<Integer> p) – rimuove tutti gli elementi per i quali il predicato è vero
  - toSet/toList() – imposta l'informazione che l'elenco sarà un insieme o una lista (lista di default)
  - repeat(k) – dato un intero k, replica la lista k volte (applicabile solo se la collezione risulterà essere una lista)
  - build() – restituisce l'oggetto così configurato
- L'esercizio si può svolgere sia costruendo l'oggetto man mano (stile StringBuilder) sia configurando il builder per poi costruire l'oggetto (la lista) solo nel momento in cui viene invocato il metodo build
  - toLong() – imposta l'informazione che l'elenco restituito sia di long e non più di interi (**idealmente utilizzando i generici!**)



# Esercizio con il pattern Builder: StringBuilder2

- Utilizzare il pattern Builder per creare una versione "lazy" dello StringBuilder
- Il problema è che lo StringBuilder allarga l'array dei caratteri ogni volta che viene invocato il metodo append
- Lo StringBuilder ha un metodo append che memorizza l'informazione (uno che prende in input un carattere, uno che prende in input una stringa) e un metodo toString (che è il nostro build)

# Builder




Si adotti il Builder per creare i Decorator:

il package delle classi revisionate con l'adozione del pattern ObserverObservable è "borsa_oo_singelton_factory_decorator_builder"

a cosa serve?

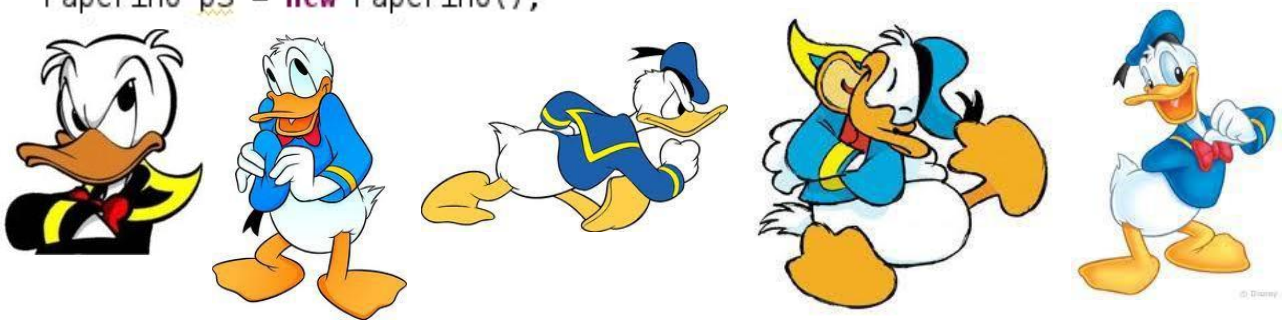
# Possono esistere classi con un'unica istanza?

- Certamente! Ad esempio: Paperino
- Tuttavia nulla mi impedisce di creare più istanze DISTINTE!

```
public class Paperino extends PersonaggioDisney
{
    public void lucidaMoneteDelloZione()
    {
        System.out.println("swish swish swish...");
    }

    public static void main(String[] args)
    {
        Paperino p1 = new Paperino();
        Paperino p2 = new Paperino();
        Paperino p3 = new Paperino();
    }
}
```

Ma questo non ha senso!



# Il Singleton Pattern: forzare un unico punto di accesso

```
public class Paperino extends PersonaggioDisney
{
    static private Paperino istanza;

    static public Paperino getInstance()
    {
        if (istanza == null) istanza = new Paperino();
        return istanza;
    }

    private Paperino()
    {
        // costruisci l'unico oggetto
    }

    public static void main(String[] args)
    {
        Paperino p = Paperino.getInstance();
        Paperino p2 = Paperino.getInstance();
        assert(p == p2);
    }
}
```

Unica istanza della classe (campo statico!)

Punto di accesso per la costruzione

Costruttore privato: nessuno può chiamarlo, tranne metodi (statici) della classe stessa



# Paperino vs. Paperinik

- Ricordate l'esercizio Disney vs. Marvel?
- Bisognava fare in modo che i personaggi avessero un'identità **pubblica** e un'identità **privata**
- Ma come fare in modo di non creare un'istanza per Paperino e un'altra istanza per Paperinik? O addirittura istanze multiple di Paperino o Paperinik?





# Paperino vs. Paperinik

- Con il pattern Singleton!

Ho un'unica istanza: di Paperinik (che è anche Paperino)

Paperino implementa il singoletto

Paperinik ha un costruttore privato, quindi inaccessibile all'esterno



```java
/**
 * Paperino e Paperinik sono un'unica entità:
 * Questo è possibile solo con il Singleton Pattern!
 *
 * @author navigli
 */
public class Paperino extends Personaggio implements DoppiaVita
{
    /**
     * Identita' segreta
     */
    static private Paperinik paperinik;

    static public Paperino getInstance()
    {
        if (paperinik == null) paperinik = new Paperinik();
        return paperinik;
    }

    private Paperino() { }

    @Override
    public Supereroe assumiIdentitaSegreta() { return paperinik; }

    @Override
    public Personaggio assumiIdentitaPubblica() { return this; }

    /**
     * Implementa Paperinik
     * @author navigli
     *
     */
    static private class Paperinik extends Paperino implements Supereroe
    {
        private Paperinik() { }

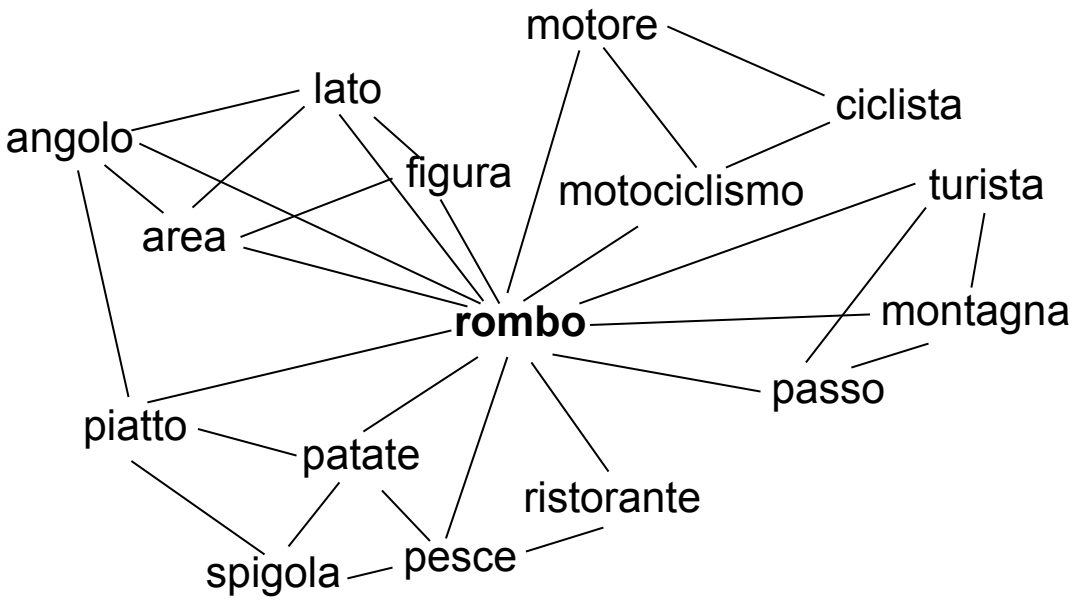
        @Override
        public Supereroe assumiIdentitaSegreta() { return this; }

        @Override
        public Personaggio assumiIdentitaPubblica() { return this; }

        @Override
        public void attacca(Supereroe s)
        {
            System.out.println("Potere degli stivaletti a molla!");
        }
    }
}
```

# Un altro esempio: una rete semantica

- E' un grafo che codifica la conoscenza del mondo reale
- Mediante:
  - concetti (nodi del grafo)
  - archi (relazioni semantiche tra concetti)




- La rete semantica è **unica**, per cui va rappresentata mediante un singoletto

# Che principio abbiamo utilizzato?

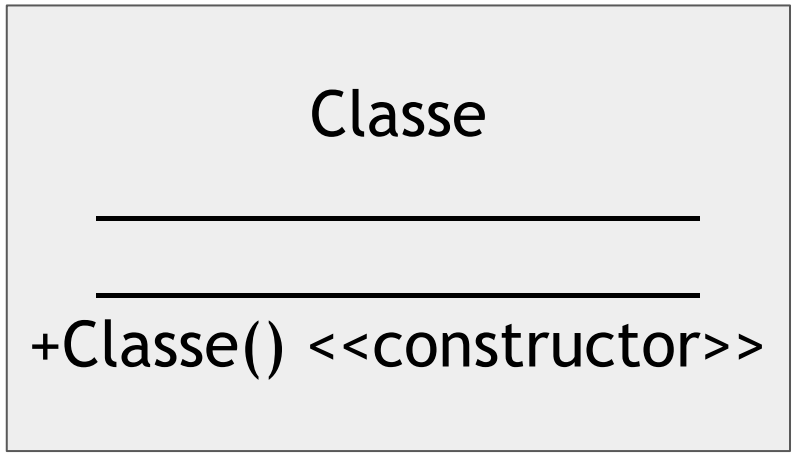
**Principio di design:**

Fai in modo che la classe modelli la realtà e **non** permetta cose concettualmente insensate

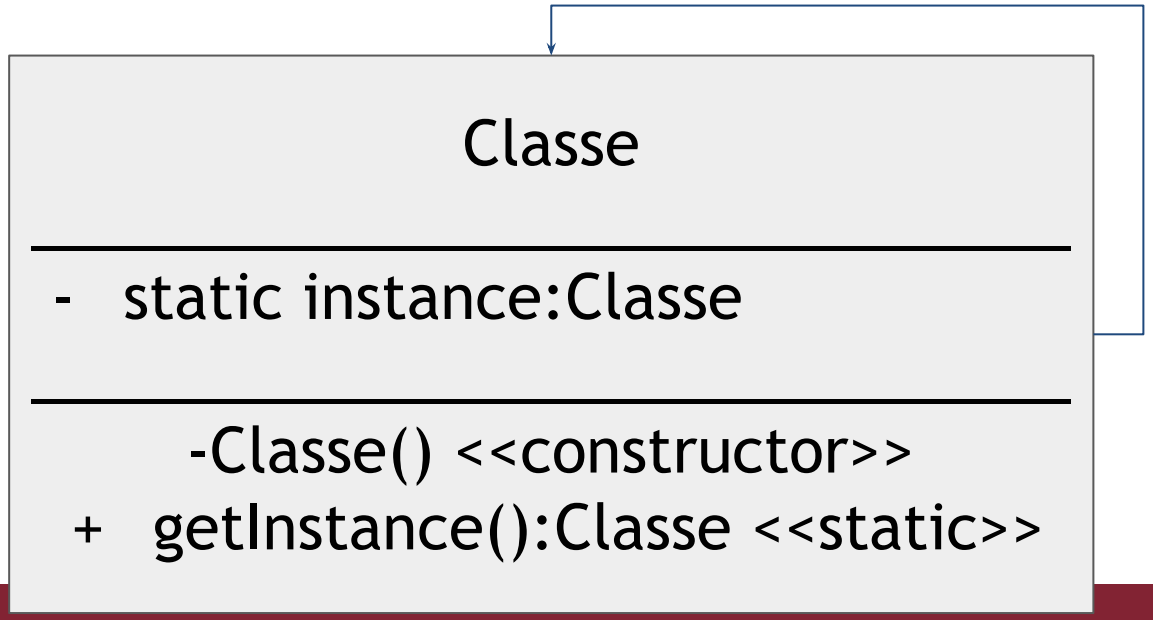


- Il Singleton Pattern permette di obbligare/vincolare la costruzione di un unico oggetto mediante un unico punto di accesso statico e un costruttore privato

# Singleton ….




serve a rendere una Classe istanziabile una ed una sola volta!!!
perché questo pattern risulta fondamentale? In quale contesto è necessario?



# Come associare diverse implementazioni a un'astrazione

- Supponiamo di modellare un oggetto Automobile mediante due classi astratte:



```
abstract public class Automobile
{
    abstract public void guida();
}
```

```
abstract public class AutomobileConTarga extends Automobile
{
    private String targa;
    private Color colore;

    public AutomobileConTarga(String targa, Color colore)
    {
        this.targa = targa;
        this.colore = colore;
    }

    public String getTarga() { return targa; }
    public Color getColore() { return colore; }
}
```

# Come associare diverse implementazioni a un'astrazione

- Abbiamo due tipi concreti di automobili

```
public class Ferrari extends AutomobileConTarga
{
    public Ferrari(String targa, Color colore)
    {
        super(targa, colore);
    }

    public void guida() { System.out.println("Vrroooooooom!!!"); }
}
```



```
public class Cinquecento extends AutomobileConTarga
{
    public Cinquecento(String targa, Color colore)
    {
        super(targa, colore);
    }

    public void guida() { System.out.println("Po po po po po po po po..."); }
}
```

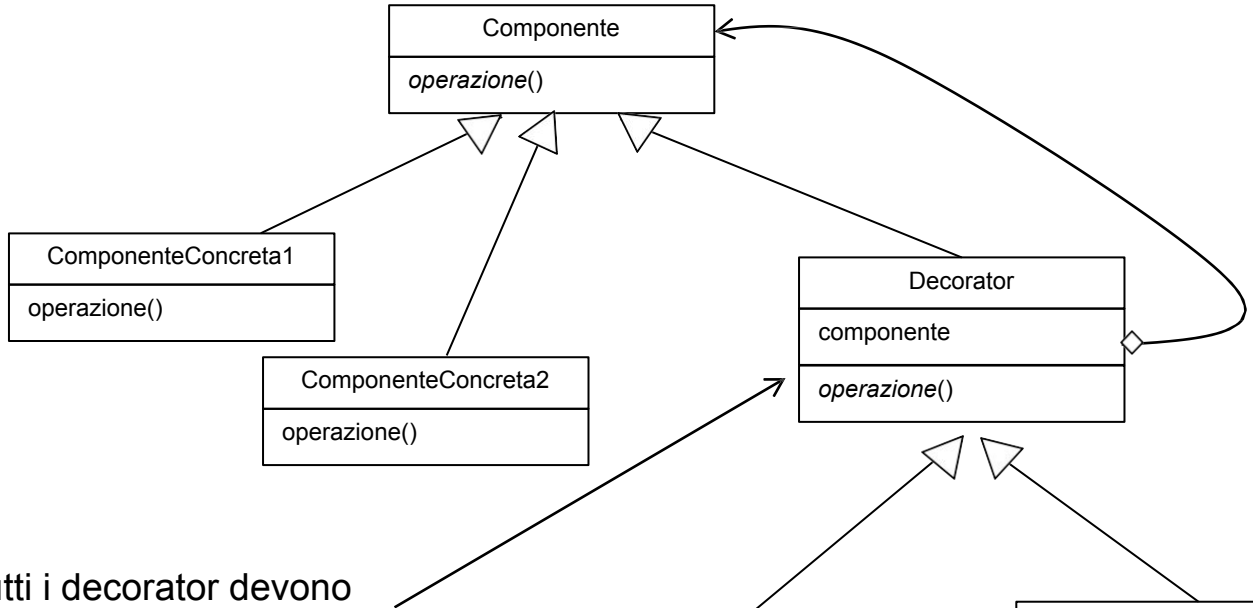


- Supponiamo di voler associare una rappresentazione grafica alle automobili

# Il Decorator Pattern

- E' possibile aggiungere nuove responsabilità a un oggetto **senza che esso lo sappia**
- Il Decorator:
  - Estende la classe astratta dell'oggetto
  - E' costruito con un'istanza concreta della classe astratta dell'oggetto
  - Inoltra le richieste di tutti i comportamenti all'oggetto (componente)
  - Effettua azioni aggiuntive (ad esempio, il disegno 2d o 3d)

# Il Decorator Pattern: The Big Picture




Tutti i decorator devono implementare l'operazione utilizzando comunque l'operazione della componente

# Il Decorator Pattern per rappresentare graficamente le automobili

```
abstract public class DecoratorAutomobile extends Automobile
{
    protected Automobile automobile;

    public DecoratorAutomobile(Automobile automobile)
    {
        this.automobile = automobile;
    }

    abstract public void guida();
}
```

Componente da decorare

Decoratore costruito con la componente da decorare

Rendiamo astratto il comportamento da “decorare”

# Il Decorator Pattern per rappresentare graficamente le automobili

```
public class DecoratorAutomobile2D extends DecoratorAutomobile
{
    private Immagine2D immagine;

    public DecoratorAutomobile2D(Automobile automobile, Immagine2D immagine)
    {
        super(automobile);
        this.immagine = immagine;
    }

    public void guida()
    {
        automobile.guida();
        immagine.setX(immagine.getX()+1);
        immagine.disegna();
    }
}
```

Incapsula la rappresentazione 2D dell'automobile

Incapsula la rappresentazione 3D dell'automobile

```
public class DecoratorAutomobile3D extends DecoratorAutomobile
{
    private Immagine3D immagine;

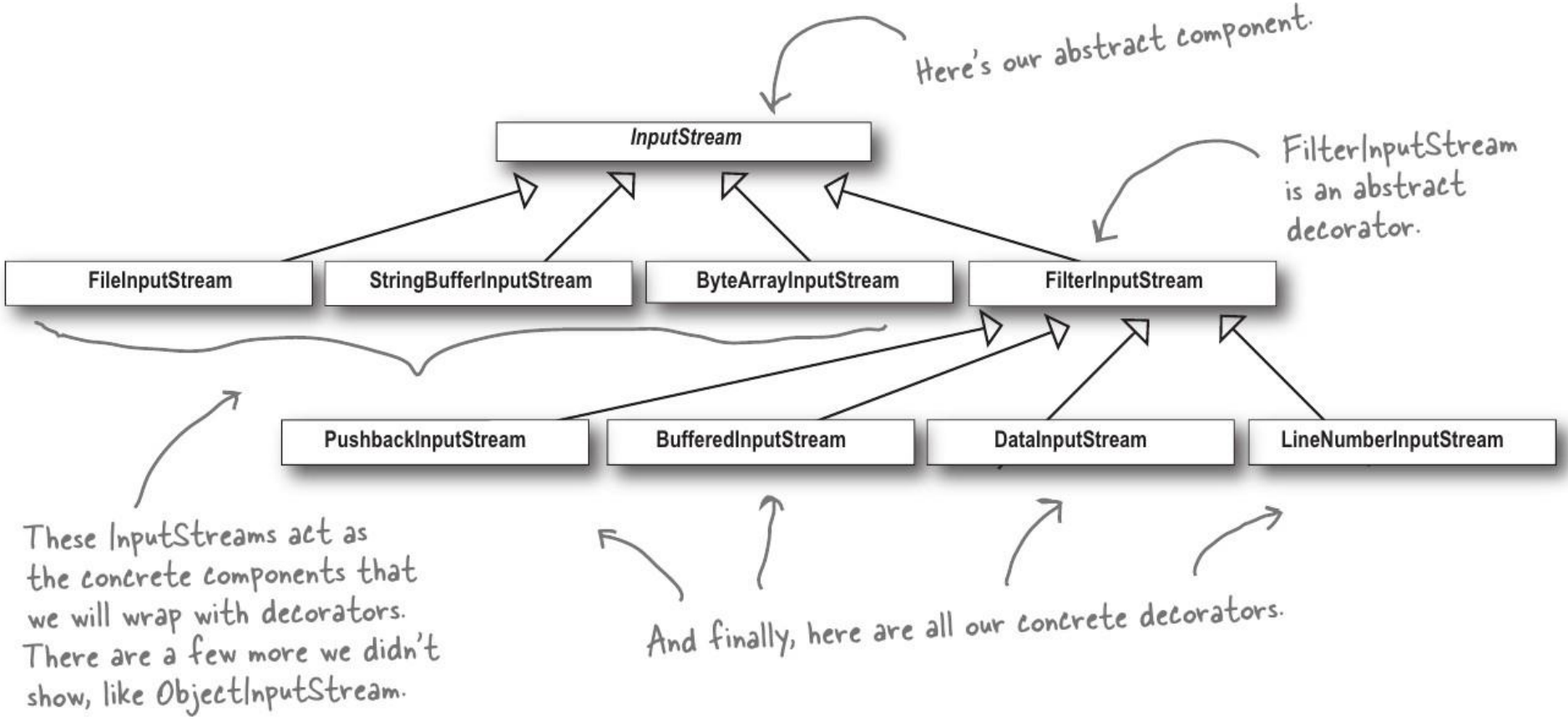
    public DecoratorAutomobile3D(Automobile automobile, Immagine3D immagine)
    {
        super(automobile);
        this.immagine = immagine;
    }

    @Override
    public void guida()
    {
        automobile.guida();
        immagine.setX(immagine.getX()+1);
        immagine.setZ(immagine.getZ()+3);
        immagine.disegna();
    }
}
```

Comportamento di base dell'oggetto decorato

Comportamento aggiuntivo del decorator

# I Decorator nelle API I/O di Java



# Che principio abbiamo utilizzato?

**Principio di design:**
Le classi dovrebbero essere aperte all'estensione ma chiuse alla modifica (Open-Closed Principle)



- Il Decorator Pattern permette di estendere le funzionalità/responsabilità di una classe, senza modificare il codice della stessa
- E' un'alternativa all'estensione mediante ereditarietà che permette la "composizione" a piacimento

# Command/Callback pattern

- A volte è necessario effettuare richieste a oggetti **senza sapere nulla relativamente all'operazione richiesta**
- L'operazione potrà eseguita **in futuro**, quando necessario
- Per fare ciò, è necessario rendere l'operazione modulare in modo che possa essere associata a un oggetto
  - E in seguito diverse associazioni possano essere fatte dinamicamente

# Come fare?

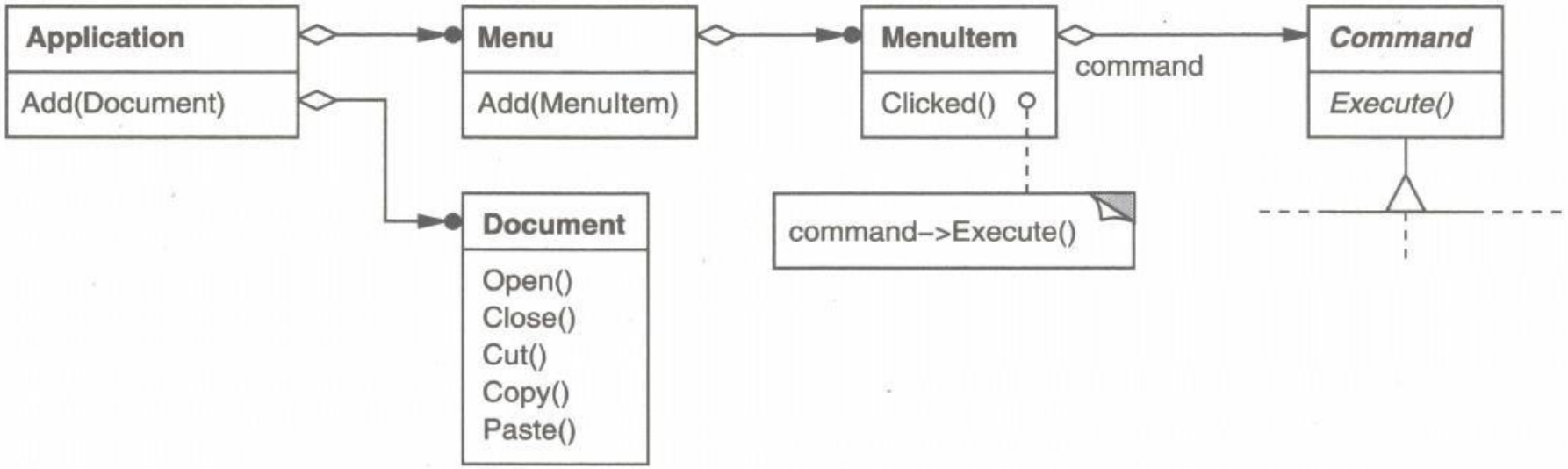
- Si crea un'interfaccia che espone il metodo generale:

```
@FunctionalInterface
public interface Callback
{
    void execute();
}
```

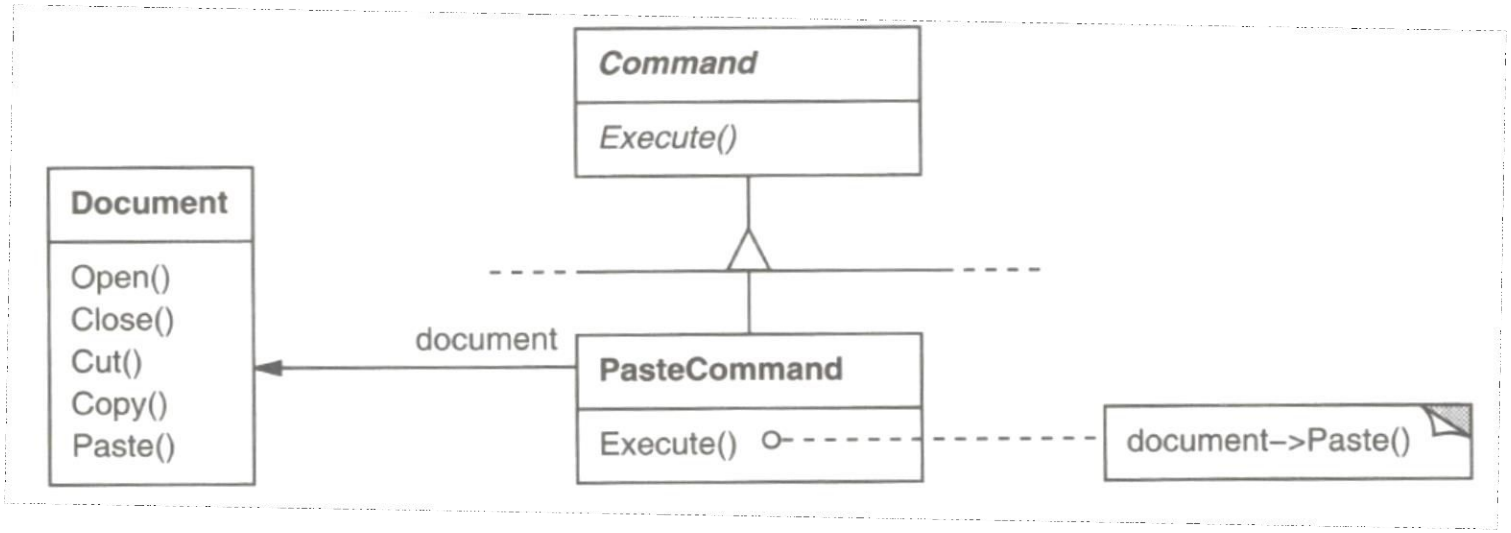
- Ogni funzione concreta implementa l'interfaccia **Callback**
- **In Java 8:**

```
Callback c = () -> System.out.println("ciao ciao!");
```

# Un esempio: associare delle operazioni a un menù



# Un esempio: associare delle operazioni a un menù





# Esercizio: applicare il pattern Command in modo da incapsulare il comportamento dell'azione associata e renderlo parametrizzato

```
abstract class Bottone
{
    abstract public void esegui();
}

class Bottone1 extends Bottone
{
    public void esegui() { System.out.println("ciao"); }
}

class Bottone2 extends Bottone
{
    private int k, j;
    private String label;
    public Bottone2(int k, int j) { this.k = k; this.j = j; }

    public void esegui() { label = ""+k/j; }
}
```

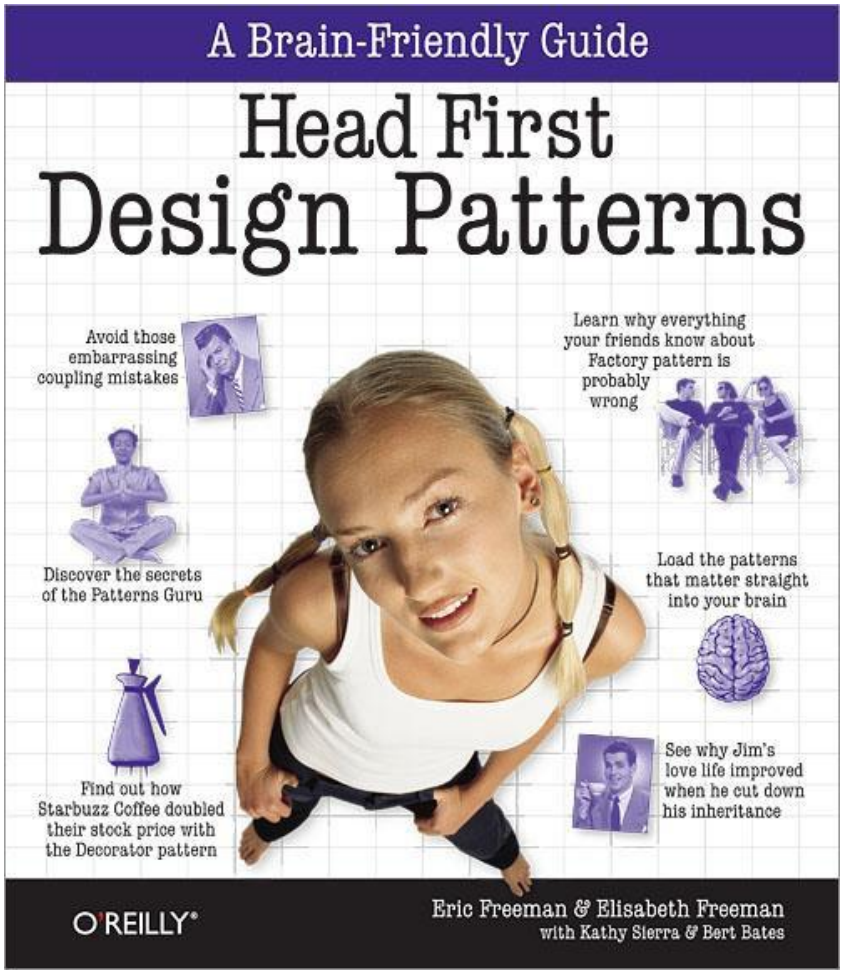
# Quali pattern abbiamo visto?

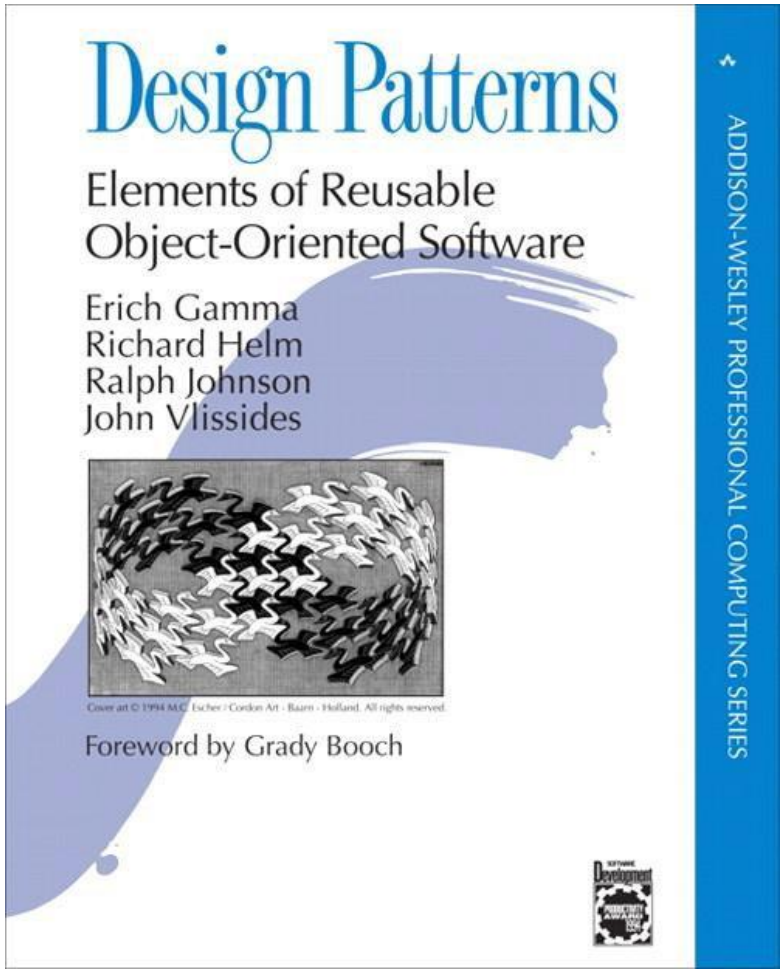
- **Behavioral pattern (relativi al comportamento):**
  - Strategy Pattern
    - Per rendere flessibile, dinamica ed estensibile la gestione dei comportamenti
  - Observer/Observable
    - Per comunicare uno stato (o evento) a oggetti interessati al suo aggiornamento (o accadimento)
  - Command
    - Per codificare funzioni da salvare in/passare a un oggetto e chiamare in seguito
  - Iterator (!!!)
    - Fornisce un modo per accedere a una collezione in modo sequenziale senza esporne la rappresentazione sottostante
  - Template method
    - Si crea un metodo astratto che si utilizza in altri metodi della classe astratta
    - Si costringono le classi concrete a fornire l'implementazione
    - Vedi esercizi ContoCorrente/Operazione bancaria, EssereVivente e Factory pattern

# Quali pattern abbiamo visto?

- **Creational pattern (relativi alla creazione di oggetti):**
  - SimpleFactory e Factory
    - Per separare la creazione degli oggetti dal resto della logica
  - Builder
    - Per "configurare" la creazione di oggetti con molti parametri, alcuni opzionali
  - Singleton
    - Per impedire la creazione di più di un'istanza di una classe
- **Structural pattern (relativi all'organizzazione delle classi):**
  - Decorator
    - Per fornire l'implementazione grafica di un oggetto

# Per approfondire…



# Credits

Le slide di questo corso sono basate su quelle utilizzate dal Prof. Faralli, le quali sono a loro volta una rielaborazione delle slide del Prof. Navigli, e frutto della revisione degli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica Mario Marra e Paolo Straniero.